DocuMind

Enterprise Document Intelligence Platform — Documentation

1. User Manual
2. Architecture & Design
3. Deployment & Operations
4. Demo deployment: Vercel + AWS Lightsail

Generated 10 November 2026 from the repository's `docs/` folder.

DocuMind User Manual

DocuMind is a document intelligence platform for an organization. You upload PDF, Word and Excel files. DocuMind reads them, indexes them for search, and answers questions about them with citations that link to the exact page, section or spreadsheet row. Every answer is limited to the documents the person asking is allowed to see.

This manual has three parts:

- **Part 1 – Using DocuMind** (sections 1–12): for everyone who uses the web app.
- **Part 2 – How it works** (sections 13–18): the technical design behind each feature.
- **Part 3 – Running and configuring** (sections 19–24): for whoever installs and operates it.

Related documents: architecture.md (design decisions in depth) and deployment.md (production deployment, backups, upgrades).

Contents

Part 1 – Using DocuMind

1. Getting started
2. Key concepts
3. Home
4. Documents
5. AI Chat
6. Compare
7. Evaluations
8. Analytics
9. Embeddings
10. User Management
11. Settings
12. Roles and permissions reference

Part 2 – How it works

13. Architecture
14. Ingestion pipeline
15. Retrieval and answers
16. Comparison
17. Evaluation metrics
18. Security model

Part 3 – Running and configuring

19. Running DocuMind
20. Configuration reference
21. Command-line tools
22. REST API
23. Troubleshooting

24. Glossary

Part 1 – Using DocuMind

1. Getting started

Create a workspace

DocuMind has no built-in default account. The first person creates a workspace:

1. Open the app. Locally this is http://localhost:3000; in a deployment it is `https://<your domain>`.
2. On the sign-in page, choose **register**.
3. Enter your full name, e-mail, a password and an organization name.
4. You are signed in as the **Admin** of the new organization.

Password rules:

- At least **10 characters** by default (set by `PASSWORD_MIN_LENGTH`).
- At least one letter and at least one digit.
- No leading or trailing spaces.

Add your colleagues

Colleagues do not register themselves into your workspace; registering always creates a *new* organization. Instead, an admin adds them:

1. Go to **User Management** → **Add user**.
2. Enter their name, e-mail, role and an initial password.
3. Share the password with them securely. They can change it later in **Settings** → **Password**.

Signing in and sessions

- You stay signed in for up to 7 days of inactivity.
- Your session renews itself in the background.
- Signing out, or changing your password, ends the session. A password change ends it on every device.
- After 5 wrong passwords in a row the account is locked for 15 minutes.

---

2. Key concepts

Organization

Everything (users, documents, chats, evaluations) belongs to one organization. Organizations are fully isolated: nobody can see another organization's data, and a document id from another organization behaves as if it doesn't exist.

Roles

Each user has exactly one role:

Role	Intended for	In short
Admin	Workspace owners, IT	Everything: all documents, users, groups, settings, analytics, embeddings, evaluations
Manager	Team leads, content owners	Uploads documents, manages categories, views evaluation results
User	Everyone else	Reads documents shared with them, chats, compares. Can upload only if an admin allows it

The full matrix is in section 12.

Document visibility

Each document has one of three visibility levels:

Visibility	Who can see it
Everyone in the organization	All members
Restricted (shared only)	The owner, admins, and the people or groups it is explicitly shared with
Private (me and admins)	The owner and admins only. Sharing has no effect

Rules that apply on top of visibility:

- Sharing gives either **Can view** or **Can manage**. *Manage* lets that person edit, re-index, share and delete the document.
- Visibility applies everywhere: in the document list, in search, in chat answers, in comparisons and in evaluation results. The AI cannot quote a document you aren't allowed to open.

Document status

Every upload moves through these states:

```
uploaded ──▶ processing ──▶ indexed      (searchable, usable in chat)
                        └──▶ failed      (reason shown on the document's Details tab)
```

Only **indexed** documents are used for answers.

Categories and groups

- **Categories** label documents (for example Finance or Legal) for filtering.
- **Groups** are teams of people. You can share a *restricted* document with a whole group at once.

---

3. Home

The home page shows:

- Counts of total, indexed, processing and failed documents.
- An **Ask your documents** box that starts a chat.
- Your recent documents with their processing status.
- Your recent conversations.

---

4. Documents

Uploading

1. On **Documents**, choose **Upload documents**.
2. Drag files in, or click to browse.
3. Choose **Who can see these documents** (visibility) and, optionally, a **Category**.
4. Upload. Files are sent in parallel, three at a time, each in its own request with its own progress bar, so a large file doesn't hold back the smaller ones. Each file appears in the list as soon as its own upload finishes, with status *uploaded*, and moves to *indexed* on its own; the page updates live. You can add more files while others upload. You can also close the dialog: uploads in progress continue, and a notification reports the result.

Upload rules:

Rule	Default
File types	PDF, DOCX, XLSX, XLS
Maximum file size	50 MB (`MAX_UPLOAD_SIZE_MB`)
Files per upload	20 (`MAX_FILES_PER_UPLOAD`)
PDF pages	2,000 (`MAX_PDF_PAGES`)
Spreadsheet rows	100,000 per sheet (`MAX_SHEET_ROWS`)

What DocuMind checks:

- **Content, not just the extension.** A renamed file is rejected.
- **No macros.** Macro-enabled Office files are refused.
- **Duplicates.** Uploading an identical file again is reported as a duplicate.
- **Unusable files.** Encrypted or corrupt files, and scanned PDFs with no text layer, are reported as failures with the reason. DocuMind does not perform OCR.

Who can upload: managers and admins. Users can upload only when an admin turns on **Settings** → **Users can upload documents**.

Finding documents

The list can be searched by title or file name, and filtered by status, file type and category. It is paginated.

Document detail page

Click a document to open it. The page has three tabs:

Extracted content

- The text DocuMind extracted, split into the same chunks it uses for answers.
- Each chunk shows its location: page, section heading path, or sheet and rows.

Preview

- PDFs render inside the app.
- Other file types can be downloaded with **Download**.

Details

- *Metadata*: type, size, pages or sheets, author and similar fields.
- *Extraction notes*, for example "page 4 has no text layer".
- *Visible to*: who can see the document.
- *Processing history*: every ingestion job with its status, duration and error. If processing failed, the reason is shown here.

Actions (shown to people who can manage the document):

Action	What it does
Ask about this document	Starts a chat limited to this document
Download	Downloads the original file
Share	Opens the share dialog (see below)
Visibility	Changes between organization, restricted and private
Reprocess	Re-indexes the document, for example after a failure or a configuration change. The previous index keeps answering until the new one is complete
Delete	Removes the document. It disappears from search and chat immediately; its vectors are then purged

Sharing

Share opens a dialog:

1. Add a person or a group.
2. Pick **Can view** or **Can manage**.
3. Remove an entry to revoke it.

Sharing only has effect when visibility is **Restricted** (or, for *Can manage* grants, also **Everyone in the organization**). Changes apply to search and chat immediately.

---

5. AI Chat

Asking a question

1. Open **AI Chat** and type a question, for example "*How long are customer database backups kept?*".
2. While the answer is prepared you see the stages: *Thinking* (DocuMind decides whether your message needs your documents), then, for document questions, *Searching your documents* and *Reading sources*. The answer then streams in word by word.
3. Use **Stop** to interrupt an answer.

Conversations are saved in the list on the left. They are private to you. Follow-up questions use the last few turns as context, so "*and for the HR system?*" works.

Chatting naturally

You don't have to start with a question about a file. DocuMind decides for each message whether it needs your documents:

You say	DocuMind does
A greeting, thanks, goodbye ("Hello!", "Thanks!")	Replies in a sentence or two. No search, no citations
"What can you do?", "Who are you?"	Explains briefly what it can help with
A general question or writing help ("Explain what a REST API is")	Answers from general knowledge and says so; it is **not** from your documents
Anything about your organization's rules, figures, files or people	Searches your documents and answers with citations

Replies that need no documents carry no citations and no warning badge. When a *document* question can't be backed by a citation, the answer shows **Not backed by document citations**.

When DocuMind is unsure, it searches your documents; it never answers a question about your organization from memory. If you ever get a general answer to a question that should have used your documents, ask again naming the topic or document.

Choosing a model

The model picker sits at the bottom left of the message box. It lists every model your administrator has made available (those whose provider key is configured), grouped by provider, with its price per million input and output tokens. The choice is remembered in your browser and applies to your next message, so you can switch models in the middle of a conversation: the history is plain text and works with any model.

- A model that just failed because its quota or usage limit is used up is greyed out in the list, with the provider's explanation (for example when access returns). It becomes selectable again once it works, or after ten minutes.
- Cheaper models suit simple questions; larger ones help with hard reasoning. Document answers are grounded in the same retrieved passages whichever model writes them.

Editing and resubmitting a message

Point at one of your messages and choose the pencil icon. Change the text (or leave it as it is to ask again with a different model) and choose **Resubmit**.

- Your message and every reply after it are replaced by the new answer. The answer is written using only the conversation *before* the edited message.
- Nothing is deleted: the replaced messages are kept out of sight in the database (for the audit trail), but they no longer appear and are not sent to the model.
- An edited message is marked *edited*. If you edit the first message, the conversation is renamed after it.
- You can only edit your own messages, and not while a reply is being written.

What a reply costs

Under every reply a line shows the model, the tokens and the **estimated cost**, for example *Gemini 3.5 Flash-Lite · 1,529 in · 124 out · $0.0004*. Point at it for the breakdown:

- **Answer**: the model you chose. Tokens *in* are your question, the conversation and any retrieved passages; tokens *out* are the reply (for reasoning models this includes their hidden thinking).
- **Routing**: the small call that decides whether your message needs your documents. It runs on the default (or configured routing) model, not the one you picked, and is skipped for plain greetings.
- How long the reply took, and cached or reasoning tokens where the provider reports them.

The conversation header shows the running total. Costs are **estimates**: provider-reported token counts multiplied by the prices in the model catalog. They are saved with each reply, so changing a price later does not rewrite history. If a model has no price in the catalog you see *cost n/a* and only tokens.

Citations

Answers contain numbered markers like **[1]**. Each marker is a citation with:

- the document title;
- the location: page, section, or sheet and row numbers;
- a quoted snippet.

Click a citation to open the source document. DocuMind checks every citation against the passage it claims to quote; a citation that doesn't match is dropped and never shown.

When the documents don't say

If no document is relevant enough, DocuMind says it couldn't find the answer instead of guessing, and suggests rephrasing or naming the document. In that case the language model isn't called at all.

When an answer adds reasoning that goes beyond what the documents state, that part is labelled **Interpretation:**, so facts and inference stay separate.

Limiting a chat to certain documents

Ask about this document on a document page starts a chat limited to that one document. The chat header then shows *Scoped to 1 document(s)*.

Through the API, a conversation or a single message can be scoped to up to 100 documents and 50 categories. A scope can only narrow what you're allowed to see, never widen it.

---

6. Compare

Compare puts 2–5 documents side by side.

1. Choose **New comparison**.
2. Add 2–5 indexed documents.
3. Optionally enter a **focus question**, for example *"How do retention periods differ?"*.

The result is a table:

- **Rows** are aspects, such as *retention period* or *approval process*.
- **Columns** are documents.
- Each cell is **present** (with the document's own cited evidence), **absent** or **unclear**.
- Each aspect has a short summary of the differences. Aspects where the documents conflict are flagged under **Missing or inconsistent**.

Rules:

- A "present" finding without valid evidence from *that* document is downgraded to "unclear".
- You can only compare documents you are allowed to view.
- Past comparisons are listed under **History**.

7. Evaluations

Evaluations measure how well DocuMind answers questions about your own documents, so you can tell whether a change (a new model, different chunking or a new prompt) made it better or worse. Managers and admins can see results; only admins can start runs.

Datasets and test cases

1. **New dataset**: give it a name and description.
2. Open it and choose **Add a test case**:
    - **Question** (required).
    - **Reference answer** (optional): the correct answer, used to grade correctness.
    - **Relevant documents**: the documents that contain the answer. Leave this empty for a question the documents *cannot* answer. The test then checks that DocuMind correctly says "not found".
    - **Evidence quote** (optional, recommended): exact text from the document that answers the question. It enables passage-level metrics and still works after the document is re-chunked.
    - **Tags**, comma separated.
3. To add many cases at once, use **Import JSON** with an array of cases:

```
[
  {
    "question": "How long are database backups kept?",
    "expected_answer": "Thirty days.",
    "relevant_document_ids": ["<document uuid>"],
    "evidence": [{ "document_id": "<document uuid>", "quote": "kept for thirty days" }],
    "tags": ["retention"]
  },
  { "question": "What is the CEO's favourite colour?", "relevant_document_ids": [] }
]
```

Limits: up to 20 relevant documents, 20 evidence quotes and 10 tags per case.

Running and reading results

- **Run** executes every case through the real search-and-answer pipeline, as the admin who started it. It runs in the background.
- The run page shows:
    - **Metrics**: averages, with the change against the previous run. Drops greater than 5 points are flagged.
    - **Per-case results**: answer, citations, faithful, correct, abstention, latency, and the judge's rationale and unsupported claims.
    - **Configuration**: everything that can affect results, recorded with the run (models, chunk sizes, thresholds, prompt and judge versions).
- The dataset page shows a **Quality trend** across completed runs.

What each metric means is described in section 17.

Redaction: if a test case or a run's answer involves a document you aren't allowed to view, the reference answer, quotes, answer, citations and judge notes are hidden from you; the scores remain visible. The page tells you when this has happened.

---

8. Analytics

Admins only. Choose a time range, for example **Last 7 days**. The page shows:

- **Tiles**: documents, indexed chunks, questions and grounded answers. Conversational replies (greetings and the like) are not document answers, so they don't count towards the grounded rate or the answer latency.
- **Document activity**: uploads and successful indexing per day, plus failures.
- **Questions per day**, with the "not found" rate.
- **Retrieval latency** per stage (search, rerank, generation), as p50 and p95.
- **Recent document activity**.

All figures come from the live database; nothing is sampled or simulated.

---

9. Embeddings

Admins only. An *embedding configuration* is the combination of embedding model and chunking settings used to index documents. Changing any of them on the server creates a new configuration and a new index.

The page shows:

- How many documents are on the active index, are **stale** (on an older configuration), are not yet indexed, or have failed.
- Each configuration with its chunking settings and its document, chunk and vector counts. A warning appears if the vector count doesn't match the chunk count.

Re-index actions: **stale**, **failed** or **all** documents. Each document keeps answering from its old index until its new version is complete, so re-indexing causes no downtime.

---

10. User Management

Admins only.

- Search people by name or e-mail and see each person's role and last sign-in.
- **Add user**: full name, e-mail, role and an initial password.
- Change a person's role, or use the active switch to deactivate them. Role changes take effect on that person's next request. Deactivating someone signs them out everywhere.

---

11. Settings

Section	Who	What
Organization	Admin	Workspace name; **Users can upload documents** (off by default); **Default visibility for new uploads** (uploaders can still choose for each upload)

Section	Who	What
Categories	Admin, Manager	Create, rename and delete categories
Groups	Admin	Create groups and choose their members
Password	Everyone	Change your own password. This signs you out on every device

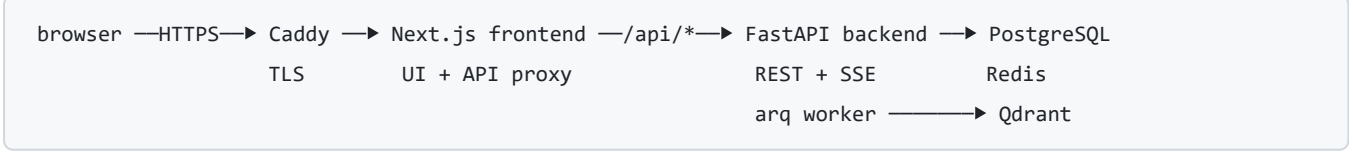
12. Roles and permissions reference

Capability	Admin	Manager	User
View documents shared with them or visible to the organization	✓	✓	✓
View and manage *every* document, including private ones	✓		
Upload documents	✓	✓	only if enabled in Settings
Manage own uploads (edit, share, re-index, delete)	✓	✓	✓
AI Chat, search, Compare	✓	✓	✓
Manage categories	✓	✓	
View evaluations	✓	✓	
Create datasets and run evaluations	✓		
Analytics	✓		
Embeddings and re-index everything	✓		
Manage users and groups	✓		
Organization settings	✓		

The internal permission names are `documents:admin`, `documents:upload`, `categories:manage`, `evaluations:view`, `evaluations:run`, `analytics:organization`, `organization:manage`, `users:manage`, `groups:manage` and `audit:view`. The backend checks these permissions, never role names, and the navigation hides pages you can't use.

Part 2 – How it works

13. Architecture



Component	Technology	Role
Frontend	Next.js 16, React, Tailwind, shadcn/ui, TanStack Query	The UI. It proxies `/api/*` to the backend, so cookies stay first-party and no CORS is needed

Component	Technology	Role
Backend	FastAPI, async SQLAlchemy 2, Pydantic v2	REST API, authentication, permissions, search, chat streaming (SSE)
Worker	arq (Redis queue)	Parsing, chunking, embedding and indexing documents; evaluation runs; a sweep for stuck jobs
PostgreSQL	16, Alembic migrations	Source of truth: users, documents, chunks, conversations, evaluations, audit log
Qdrant	1.20	Dense and sparse vectors with permission (ACL) data attached
Redis	7	Job queue and rate limiting
Caddy	2	TLS, real client IP, the only publicly exposed service
LLM	OpenAI, Anthropic Claude and Google Gemini (several can be enabled at once)	Answer generation, routing, comparison, evaluation judging

Embeddings and reranking run **locally on the CPU** with fastembed (ONNX). No document text leaves the server for indexing. Only the passages selected for an answer are sent to the LLM provider.

Backend layout: `app/api` (routers and dependencies), `app/services` (use cases, access policy), `app/repositories` (organization-scoped queries), `app/models`, `app/schemas`, `app/workers`, `app/core` (config, security, logging, rate limiting).

14. Ingestion pipeline

Triggered by every upload or re-index. The worker:

1. **Parses** the file:
   - **PDF** with pdfplumber: page text, tables converted to Markdown, metadata, and detection of scanned pages.
   - **DOCX** with python-docx: the heading hierarchy becomes section paths; tables are kept.
   - **XLSX** with openpyxl and **XLS** with xlrd: one `Header: value` record per row.
2. **Chunks** the text with LangChain's `RecursiveCharacterTextSplitter`, measured in model tokens:
   - Default size 400 tokens with 60 tokens of overlap.
   - Chunks never cross a page, section or sheet boundary.
   - Table and row-group chunks repeat their header row.
3. **Embeds** each chunk with `BAAI/bge-base-en-v1.5` (768-dimension dense vectors) and `Qdrant/bm25` (sparse keyword vectors).
4. **Indexes** the chunks in Qdrant. There is one collection per embedding configuration (`dm_<model>_<chunking>_<hash>`), reached through the alias `chunks_active`. Each point's payload holds:
   - the organization;
   - ACL tokens;
   - page, section, sheet and rows;
   - the chunk text.
5. **Publishes** the new version. New points are written as `searchable=false` and switched on only when the whole version is stored, so the old version keeps answering until then.

Additional behaviour:

- Chunks that look like prompt injection (for example "ignore previous instructions") are flagged.

- Transient errors are retried up to 3 times with exponential backoff (`INGESTION_MAX_TRIES`). Permanent errors (encrypted, corrupt, or no text) fail immediately with a clear reason.
- A sweep job every 5 minutes re-queues lost jobs and recovers stuck ones.

15. Retrieval and answers

For each message:

0. **Route** (`app/rag/routing.py`). Unmistakable greetings, thanks and goodbyes are recognised by rule, with no model call. Anything else gets one small structured-output call (low reasoning effort) that returns `chat` or `documents` and, for follow-ups, a standalone search query (this replaces the former separate query-rewrite call).
    - **chat**: the assistant replies using `ASSISTANT_SYSTEM_PROMPT` with no document access, no search and no citations. The reply never states facts about the organization.
    - **documents**: steps 1-5 below.
    - If the routing call fails, or the model is unsure, the message is treated as `documents` .
1. **Retrieve.** One Qdrant query combines dense and BM25 search (up to 40 candidates each), fused with Reciprocal Rank Fusion. The permission filter (organization, `searchable` , ACL tokens, optional scope) is built from the authenticated user on the server, never from request input.
2. **Re-check.** Every hit is verified again in PostgreSQL: the document must still exist, the user must still be allowed to view it, and the chunk must belong to the active index version. Stale vector data can therefore only remove results, never add them.
3. **Rerank.** The top 20 hits are rescored by the `BAAI/bge-reranker-base` cross-encoder.
    - If the best score is below `RETRIEVAL_MIN_RELEVANCE` (0.002), the answer is "not found" and the LLM is not called.
    - Otherwise the top 8 chunks are kept, at most 4 per document, within a 6,000-token budget.
4. **Generate.** The answer depends on `LLM_PROVIDER` :
    - **OpenAI**: the Responses API with `store=false` . Sources are numbered `<source>` blocks, cited with `[n]` markers (one citation per chunk).
    - **Anthropic**: Claude's native citations over one document block per sentence or row, so citations point at exact passages.

    Either way, the answer streams over Server-Sent Events. The stream sends these events in order: `meta` , `status` (`thinking` , then `retrieving` for document questions, then `generating`), `sources` , `token` , `citation` , `done` (or `error`). `done` carries `route` (`chat` or `documents`), and each stored message records its route. 5. **Validate citations.** Each citation is checked against the chunk it references. Invalid ones are dropped and counted.

Retrieved text is treated as data, never as instructions: the model has no tools, and flagged chunks are marked as suspicious in the prompt.

Prompts (`app/rag/generation/prompts.py` , `PROMPT_VERSION` is recorded with every evaluation run):

- `ASSISTANT_SYSTEM_PROMPT` : the personality (warm, brief, natural, in the user's language), what DocuMind can and cannot do, and the honesty rules. It is the voice of every reply.
- `ANSWER_SYSTEM_PROMPT` / `MARKER_ANSWER_SYSTEM_PROMPT` : the same persona plus the rules for answering from excerpts (cite, say so when the documents don't cover it, keep **Interpretation:** apart from facts). Native citations for Claude, `[n]` markers for OpenAI.

- `ROUTER_SYSTEM_PROMPT`: the chat-or-documents decision and the query rewrite.
- `JUDGE_SYSTEM_PROMPT` (`app/rag/evaluation/judge.py`, `JUDGE_VERSION`): the evaluation rubric.

Measured retrieval quality on the built-in benchmark (12 paraphrased questions, real models): hybrid search with reranking reaches MRR 0.96 and NDCG@5 0.97.

16. Comparison

1. Each selected document contributes its own sources: chunks relevant to the focus question or, without one, its opening content. This way a large document cannot crowd out the others.
2. One structured-output call (JSON schema) returns aspects × documents with `present / absent / unclear`, a summary of the differences and a conflict flag.
3. Validation on the server:
    - Each finding keeps only citations from its own document.
    - "Present" without valid evidence becomes "unclear".
    - Labels the model invents (such as `D1`) are replaced with document titles.

17. Evaluation metrics

Group	Metric	Meaning
Retrieval	Precision@5, Recall@5	Share of the top 5 chunks that are relevant, and share of the relevant chunks found in the top 5
	MRR	How high the first relevant chunk ranks (1.0 = first)
	NDCG@5	Ranking quality of the top 5
	Document recall	Share of the relevant documents that were retrieved
Answer (LLM judge, rubric `judge-v2`)	Faithfulness	Claims supported by the cited sources
	Hallucination rate	Share of unsupported claims
	Answer relevance	Whether the answer addresses the question
	Context relevance	Whether the retrieved passages fit the question
	Correctness	Agreement with the reference answer
Deterministic	Citation accuracy	Citations that match their passages
	Abstention accuracy	Says "not found" exactly when it should
	Latency	Time for retrieval and answer

How metrics are computed:

- The rubric (`JUDGE_SYSTEM_PROMPT`) ignores friendly wording, offers of help and citation markers, counts only statements of fact about the documents as claims, and defines each 1-5 score. Evaluation runs test the document-answer path; routing is covered by its own tests.
- The judge uses `LLM_MODEL`, unless `EVAL_JUDGE_MODEL` names a different model from the same provider. A stronger judge model reduces self-preference bias.
- Relevance labels come from the evidence quotes, which are resolved against the chunks of the configuration being tested. This lets one dataset compare different chunking settings fairly.

18. Security model

Authentication

- Passwords are hashed with Argon2id.
- Access tokens are 15-minute JWTs, held in memory by the browser.
- Refresh tokens:
  - 7-day opaque tokens, in an `httpOnly`, `Secure`, `SameSite=Strict` cookie scoped to `/api/v1/auth`;
  - rotated on every use and stored only as SHA-256 hashes;
  - reuse of an old token revokes the whole session family.
- Sign-in failures return generic errors, and accounts lock after repeated failures.

Authorization

- Roles map to permissions.
- The user's identity and permissions are rebuilt from the database on every request, so role changes and deactivation apply immediately.

Document access

- One policy, implemented three times and tested against each other: in Python (`can_view` / `can_manage`), in SQL for listings, and as organization-qualified ACL tokens in Qdrant.
- Another organization's ids return 404.

Evaluations

- Content drawn from documents a viewer cannot see is redacted for that viewer.

Uploads

- An extension allow-list plus content sniffing.
- Macro rejection and a zip-bomb guard (500 MB uncompressed limit).
- Size and count limits enforced before parsing.
- Files are stored under random keys, outside any web root.

Network

- Only Caddy publishes ports (80 and 443).
- Caddy replaces any `X-Forwarded-For` header sent by a client. Because only the frontend can reach the backend, the backend can trust that header for per-IP rate limits and audit IPs.

Rate limits (per minute)

- Login and register: 10 per IP, and 10 per target account.
- Token refresh: 60. Uploads: 60 (one file per request). Chat: 20. Search: 60. Everything else: 300.

Headers

- Content-Security-Policy with no third-party origins.
- `X-Frame-Options: DENY`, `nosniff`, `Referrer-Policy: no-referrer`, `Permissions-Policy`.
- HSTS in production.

Production guards

- The API refuses to start with non-secure cookies, wildcard CORS, the example JWT secret, or no Qdrant API key.
- API docs are disabled in production.

Audit log

- Records sign-ins (including failures), user and role changes, uploads, downloads, deletions, sharing and settings changes, and evaluation runs.
- Logs never contain document text, prompts, tokens or passwords.

Part 3 – Running and configuring

19. Running DocuMind

Option A – Docker Compose (production-like)

```
cp .env.example .env    # set POSTGRES_PASSWORD (hex), JWT_SECRET_KEY, QDRANT_API_KEY, SITE_ADDRESS, LLM
key
docker compose -f infrastructure/docker-compose.yml --env-file .env up -d --build
```

Open `https://<SITE_ADDRESS>`. Migrations run automatically before the backend starts. TLS, backups, upgrades and scaling are covered in deployment.md.

Option B – Native (development; this is how the Windows machine runs it)

Requirements: Python 3.12+, Node 20.9+ (22 recommended), PostgreSQL 15+, Redis 6+ and Qdrant 1.20.

Start each service in its own terminal, with `.env` at the repository root:

```
# 1. Qdrant (native binary in infrastructure/qdrant/bin)
powershell -ExecutionPolicy Bypass -File infrastructure\qdrant\start-qdrant.ps1

# 2. API
cd backend; .venv\Scripts\activate
alembic upgrade head
uvicorn app.main:create_app --factory --host 127.0.0.1 --port 8010

# 3. Worker (needed for uploads to be indexed, and for evaluations)
cd backend; .venv\Scripts\activate
arq app.workers.settings.WorkerSettings

# 4. Frontend
cd frontend
npm run dev        # http://localhost:3000; proxies /api/* to API_URL (default http://127.0.0.1:8010)
```

On its first start the worker downloads the models (about 220 MB, plus the reranker) into `MODEL_CACHE_DIR`.

Health endpoints:

- `GET /health`: the process is alive.
- `GET /ready`: PostgreSQL, Redis and Qdrant are reachable.

20. Configuration reference

All settings are environment variables, read from `.env`. Names are case-insensitive. Secrets have no defaults.

Required

Variable	Notes
`DATABASE_URL`	`postgresql+asyncpg://…`
`JWT_SECRET_KEY`	At least 32 random characters
`QDRANT_API_KEY`	Required in production
`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GOOGLE_API_KEY`	At least one is needed for chat, comparison and evaluation; set several to offer several providers' models in chat. Upload and search work without any

General

Variable	Default
`ENVIRONMENT`	`development` (or `test`, `production`)
`LOG_LEVEL`	`INFO`
`LOG_JSON`	`true`
`CORS_ORIGINS`	`http://localhost:3000`
`API_DOCS_ENABLED`	On, except in production
`REDIS_URL`	`redis://localhost:6379/0`
`QDRANT_URL`	`http://localhost:6333`

Authentication

Variable	Default
`ACCESS_TOKEN_TTL_MINUTES`	15
`REFRESH_TOKEN_TTL_DAYS`	7
`COOKIE_SECURE`	`true`
`LOGIN_MAX_FAILED_ATTEMPTS`	5
`LOGIN_LOCKOUT_MINUTES`	15
`REGISTRATION_ENABLED`	`true`. Set `false` to close public sign-up (nobody can create a workspace); create admins with `python -m app.cli create-admin`. The sign-in page then hides its "create a workspace" link

Variable	Default
`PASSWORD_MIN_LENGTH`	10

Uploads

Variable	Default
`STORAGE_LOCAL_PATH`	`./storage`
`MAX_UPLOAD_SIZE_MB`	50
`MAX_FILES_PER_UPLOAD`	20
`MAX_UNCOMPRESSED_OOXML_MB`	500
`DEFAULT_DOCUMENT_VISIBILITY`	`organization`

Ingestion

Changing `EMBEDDING_MODEL`, `SPARSE_MODEL`, `CHUNK_SIZE_TOKENS` or `CHUNK_OVERLAP_TOKENS` creates a new index. Re-index afterwards.

Variable	Default
`EMBEDDING_MODEL`	`BAAI/bge-base-en-v1.5`
`SPARSE_MODEL`	`Qdrant/bm25`
`CHUNK_SIZE_TOKENS`	400
`CHUNK_OVERLAP_TOKENS`	60
`MODEL_CACHE_DIR`	`./.models`
`MAX_PDF_PAGES`	2000
`MAX_SHEET_ROWS`	100000
`INGESTION_MAX_TRIES`	3
`WORKER_MAX_JOBS`	2
`WORKER_JOB_TIMEOUT_SECONDS`	900

Retrieval

Variable	Default
`RERANKER_ENABLED`	`true`
`RERANKER_MODEL`	`BAAI/bge-reranker-base`
`RETRIEVAL_CANDIDATES`	40
`RERANK_CANDIDATES`	20
`RETRIEVAL_TOP_N`	8
`RETRIEVAL_MIN_RELEVANCE`	0.002. Re-calibrate this if you change the reranker
`RETRIEVAL_MAX_CHUNKS_PER_DOCUMENT`	4

Variable	Default
`CONTEXT_MAX_TOKENS`	6000

Generation

Variable	Default
`LLM_PROVIDER`	`anthropic` (or `openai`, `google`): the provider of the default model
`LLM_MODEL`	`claude-opus-5-5` (this installation uses `gpt-5-nano`): the default model
`LLM_CATALOG_FILE`	Unset (built-in `app/rag/generation/models.json`). Your own copy adds models or updates prices; a broken file stops the API at start-up
`CHAT_MODEL_IDS`	Unset (every catalog model with a key is selectable). A comma-separated list limits what users can pick, e.g. `gpt-5-nano,gpt-5-mini` for a public demo
`LLM_ROUTER_MODEL`	Unset (routing uses the default model). A cheap, fast model here keeps routing cheap when users pick an expensive one
`LLM_EFFORT`	`medium`
`LLM_MAX_TOKENS`	8000
`LLM_TIMEOUT_SECONDS`	120
`CHAT_HISTORY_TURNS`	6
`EVAL_JUDGE_MODEL`	Unset (the answer model judges itself)

Rate limits (per minute)

Variable	Default
`RATE_LIMIT_ENABLED`	`true`
`RATE_LIMIT_AUTH_PER_MINUTE`	10
`RATE_LIMIT_LOGIN_PER_ACCOUNT_PER_MINUTE`	10
`RATE_LIMIT_REFRESH_PER_MINUTE`	60
`RATE_LIMIT_UPLOAD_PER_MINUTE`	60
`RATE_LIMIT_CHAT_PER_MINUTE`	20
`RATE_LIMIT_SEARCH_PER_MINUTE`	60
`RATE_LIMIT_DEFAULT_PER_MINUTE`	300

Docker stack only

Variable	Notes
`SITE_ADDRESS`	Your domain, or `localhost`
`HSTS_MAX_AGE`	Set it once real TLS works
`HTTP_PORT`, `HTTPS_PORT`	Published ports

Variable	Notes
`POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`	Database credentials

Frontend build

Variable	Notes
`API_URL`	Backend address, compiled into the build
`NEXT_OUTPUT=standalone`	Used for the Docker image

The backend validates these settings at start-up. It rejects, for example, chunk overlap that is not smaller than chunk size, or an OpenAI model name with `LLM_PROVIDER=anthropic`.

21. Command-line tools

Run these from `backend/` with the virtual environment active. In Docker, prefix them with `docker compose … exec backend`.

```
python -m app.cli reindex --stale     # documents not on the active embedding configuration
python -m app.cli reindex --failed    # retry failed documents
python -m app.cli reindex --all       # everything (e.g. after a parser improvement)

# Create an organization and its admin. Works while sign-up is closed; prompts for the password
# (or set DOCUMIND_ADMIN_PASSWORD for scripts)
python -m app.cli create-admin --email you@example.com --name "Your Name" --organization "Acme"

# Run an evaluation dataset; exits 1 on a regression (for CI)
python -m app.cli evaluate --dataset <uuid> \
    --fail-under retrieval.ndcg@5=0.7 --fail-under answer.faithfulness=0.9 \
    --max-drop 0.05 [--baseline <run uuid>]

python infrastructure/smoke_test.py https://<site> [--insecure] [--production]   # end-to-end check
```

22. REST API

The base path is `/api/v1`. Interactive docs are at `/docs`, outside production. Every endpoint except register, login, refresh and logout needs `Authorization: Bearer <access_token>`.

Errors share one format:

```
{ "error": { "code": "…", "message": "…", "details": {}, "request_id": "…" } }
```

The `request_id` matches the `X-Request-ID` response header and the server logs.

Area	Endpoints
Auth	`GET /auth/config` (public: is sign-up open?); `POST /auth/register`, `/auth/login`, `/auth/refresh`, `/auth/logout`, `/auth/change-password`; `GET /auth/me`

Area	Endpoints
Organization	`GET` / `PATCH /organization` ; `GET /organization/members`
Users (admin)	`GET` / `POST /users` ; `GET` / `PATCH` / `DELETE /users/{id}`
Groups	`GET` / `POST /groups` ; `GET` / `PATCH` / `DELETE /groups/{id}` ; `PUT /groups/{id}/members`
Categories	`GET` / `POST /categories` ; `PATCH` / `DELETE /categories/{id}`
Documents	`POST /documents/upload` ; `GET /documents` ; `GET` / `PATCH` / `DELETE /documents/{id}` ; `POST …/reprocess` ; `GET …/jobs` , `…/download` , `…/preview` , `…/chunks` ; `GET` / `PUT …/permissions`
Chat	`GET /chat/models` (models, prices, availability); `GET` / `POST /chat/conversations` ; `GET` / `PATCH` / `DELETE /chat/conversations/{id}` ; `POST …/{id}/messages` (SSE with `Accept: text/event-stream` , otherwise JSON; body: `content` , optional `model` , optional `edit_message_id`). Messages carry `model` , `usage` (tokens, `cost_usd` , per-step breakdown), `latency_ms` and `edit_of`
Search	`POST /search/hybrid` , `/search/semantic` , `/search/documents`
Comparisons	`POST` / `GET /comparisons` ; `GET /comparisons/{id}`
Evaluations	`GET` / `POST /evaluations/datasets` ; `DELETE …/{id}` ; `GET` / `POST …/{id}/cases` ; `DELETE …/cases/{case_id}` ; `POST /evaluations/run` ; `GET /evaluations/results` , `…/results/{id}` , `/evaluations/metrics`
Analytics (admin)	`GET /analytics/overview` , `/analytics/documents?days=` , `/analytics/retrieval?days=`
Embeddings (admin)	`GET /embeddings` ; `POST /embeddings/reindex` (`stale` , `failed` or `all`)

Example session:

```
API=http://localhost:3000/api/v1
TOKEN=$(curl -s $API/auth/login -H 'Content-Type: application/json' \
  -d '{"email":"you@example.com","password":"<password>"}' | jq -r .access_token)

curl -s $API/documents/upload -H "Authorization: Bearer $TOKEN" -F files=@policy.pdf

curl -s $API/search/hybrid -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"query":"How long are backups kept?","top_k":5}'

CONV=$(curl -s $API/chat/conversations -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{}' | jq -r .id)
curl -N $API/chat/conversations/$CONV/messages -H "Authorization: Bearer $TOKEN" \
  -H 'Accept: text/event-stream' -H 'Content-Type: application/json' \
  -d '{"content":"How long are backups kept?"}'
```

23. Troubleshooting

Symptom	Likely cause and fix
A document stays **uploaded**	The worker isn't running. Start `arq app.workers.settings.WorkerSettings`
The first upload takes several minutes	The worker is downloading its models (only once)

Symptom	Likely cause and fix
Status **failed**	Read the reason on the document's **Details** tab. Encrypted or scanned PDFs need to be fixed at the source; for transient errors, use **Reprocess**
Chat says it couldn't find the answer	No indexed document you can see is relevant enough. Check the document is *indexed* and shared with you, or rephrase the question
A model is greyed out or a reply ends with a limit message	The provider refused the request because the API key's quota, spend limit or credit is used up (the message says when access returns, if the provider tells us). Pick another model, or raise the limit with the provider. A Google key on the free tier cannot use Gemini Pro-tier models at all
cost n/a under a reply	The model is not in the catalog (or has no price there). Add it to `models.json` or `LLM_CATALOG_FILE`
A reply has no citations	Not every message needs the documents: greetings, thanks and general questions are answered without a search. A *document* question without citations shows **Not backed by document citations**
Chat errors with "Answer generation is not configured" (`llm_not_configured`)	Set `LLM_PROVIDER` , `LLM_MODEL` and the matching API key, then restart the API and worker
`/ready` fails, or Qdrant returns 401	Qdrant is down, or `QDRANT_API_KEY` differs between Qdrant and the app. Restart Qdrant with `start-qdrant.ps1`
"Too many requests" (429)	A rate limit was reached. Wait a minute
Locked out after wrong passwords	Wait 15 minutes, or ask an admin to set a new password
A page is missing from the navigation	Your role doesn't include it (see section 12)
The API refuses to start in production	A production guard failed. The error names the setting (e.g. the JWT secret or the Qdrant key)
Changed embedding or chunk settings, and old documents are "stale"	Expected. Re-index them from **Embeddings** or with `app.cli reindex --stale`

24. Glossary

Term	Meaning
Chunk	A passage of a document (about 400 tokens) that is indexed and cited
Embedding	A numeric vector representing a chunk's meaning, used for semantic search
BM25	Keyword relevance scoring, used for exact terms
Hybrid search	Semantic and keyword search combined
Reranker	A model that rescores the top candidates for precision
RAG	Retrieval-Augmented Generation: answering from retrieved passages
Grounded answer	An answer supported by valid citations
Abstention	Saying "not found" instead of guessing
ACL token	A permission marker stored with each vector so search can filter by access

Term	Meaning
Embedding configuration	The combination of embedding model and chunking settings; each one has its own index
Stale document	A document indexed under an older embedding configuration
Token	The unit models read and write (roughly three quarters of a word); providers bill per million tokens, separately for input and output
Superseded message	A message hidden after the user edited an earlier one; kept in the database, never shown or sent to the model

DocuMind — Phase 1: Architecture & Design

> Status: **Implemented** (Phases 1–7). This is the original Phase 1 design; "As implemented" notes mark where the build refined it. Section 13 records the questions that were open before implementation. Scope: Enterprise Document Intelligence Platform (upload → index → RAG chat with citations → comparison → analytics → evaluation).

---

0. Key decisions at a glance

Concern	Decision	Why
API	FastAPI (async), Pydantic v2, SQLAlchemy 2.0 async + asyncpg, Alembic	Async end-to-end; typed contracts; OpenAPI for free
Background jobs	**arq** workers on **Redis**	Async-native (shares the same async services/repositories as the API), retries + backoff, small footprint. Redis also backs rate limiting and SSE cancellation flags
Relational store	PostgreSQL 16 — **system of record** (users, docs, ACLs, chunks text, conversations, evals, audit)	Qdrant is treated as a rebuildable index, never the source of truth
Vector store	Qdrant — one collection per *embedding config*, accessed through an alias	Lets us re-embed / re-chunk side by side and switch atomically (needed for eval regression)
Hybrid search	Qdrant **named dense + sparse (BM25) vectors** on the same point, fused with **RRF** via the Query API	Keyword + semantic in one round trip, one ACL filter, no PG↔Qdrant sync for full-text
Embeddings	Pluggable `EmbeddingProvider`. Default: local `BAAI/bge-base-en-v1.5` via fastembed (768-d); OpenAI/Voyage selectable by config	Runs with zero extra API keys; swap without code changes
Reranking	Pluggable `Reranker`. Default: local cross-encoder `BAAI/bge-reranker-base`; Cohere optional	Large precision gain on top-K at low cost
LLM	Pluggable `LLMProvider` (OpenAI, Anthropic, Gemini; after Phase 7 several are live at once through a model registry and catalog, see §3.2). Default: Anthropic `claude-opus-5-5` via the official `anthropic` SDK, streaming, adaptive thinking; OpenAI adapter behind the same interface	Claude's native **Citations** feature ties every citation to a block we supplied → citations cannot reference text we didn't retrieve
Orchestration	LangChain used where it adds value: loaders/text splitters, prompt templates. Retrieval & generation are thin in-house services	Avoids framework lock-in on the security-critical path (ACL filtering, citation validation)
File storage	`StorageBackend` interface: local volume (dev) / S3-compatible (MinIO/S3 in prod)	Original files never served from a public path
Frontend	Next.js (App Router, TS), Tailwind, shadcn/ui, Lucide, TanStack Query, Recharts	As specified
Auth	Argon2id passwords; JWT access token (15 min, in memory) + rotating refresh token (7 d, httpOnly cookie, hashed in PG)	Refresh-token reuse detection; no tokens in localStorage

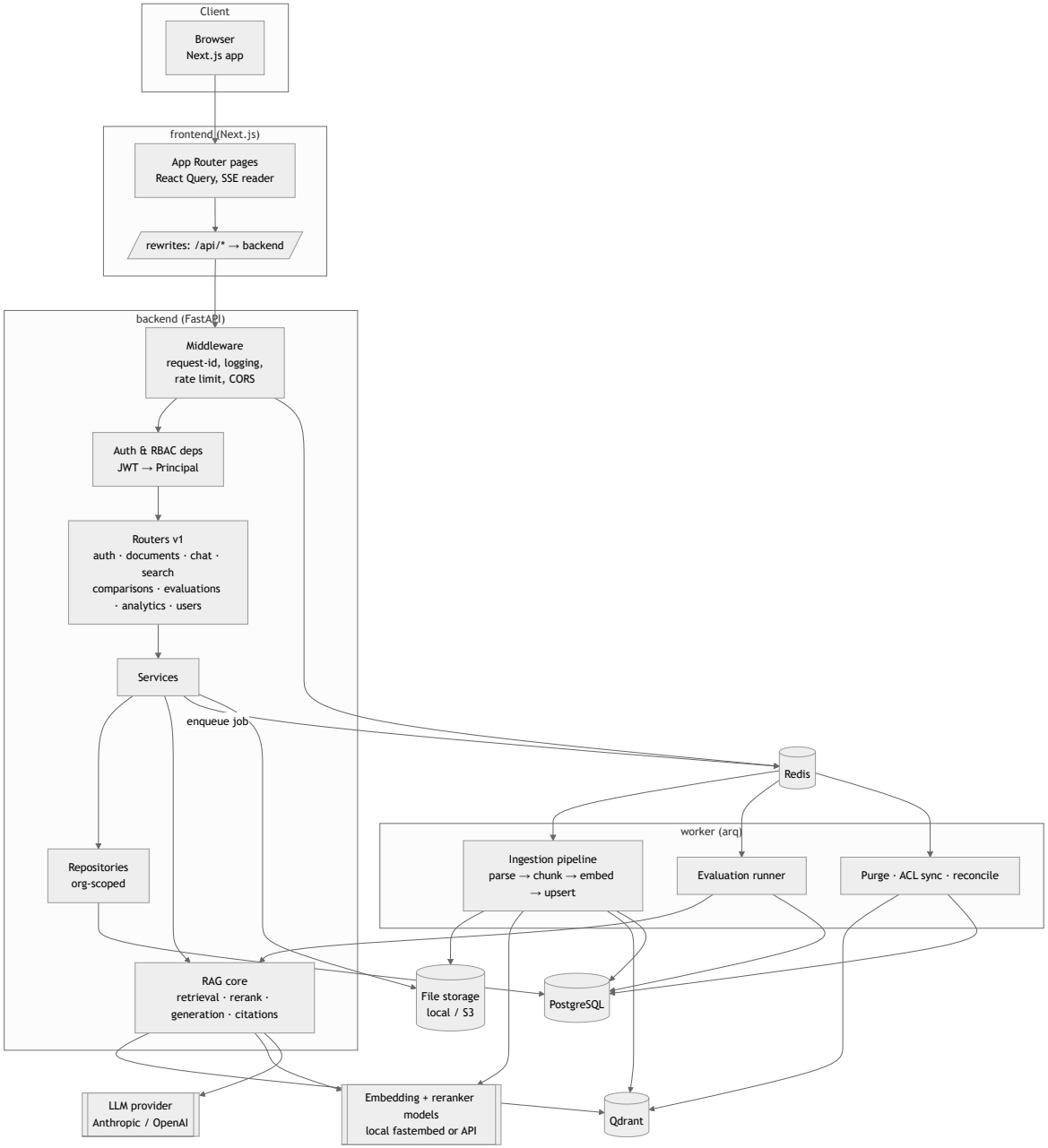
---

1. System architecture

1.1 Components & responsibilities

Component	Responsibility	Does **not** do
Next.js frontend	UI, routing, auth session handling (access token in memory, silent refresh), React Query cache, SSE stream rendering	Any authorization decision; never sees API keys
Next.js rewrite proxy	`/api/*` → FastAPI, so the browser stays same-origin (simple cookie handling, no CORS for the app)	Business logic
FastAPI API (`app/api`)	HTTP contracts, validation, authN (JWT), authZ (RBAC + document ACL), rate limiting, SSE streaming, OpenAPI	Long-running parsing/embedding
Services (`app/services`)	Use-case logic: DocumentService, ChatService, SearchService, ComparisonService, EvaluationService, AnalyticsService, AuthService	Direct SQL (goes through repositories)
Repositories (`app/repositories`)	All SQLAlchemy queries; **every query is org-scoped by construction** (`OrgScopedRepository` requires `org_id`)	Business rules
RAG package (`app/rag`)	ingestion (parsers), chunking, embeddings, retrieval, reranking, generation, evaluation — each behind a small protocol	HTTP or DB-session management
arq worker (`app/workers`)	Ingestion jobs, reprocess, vector purge on delete, ACL payload sync, evaluation runs, reconciliation	Serving requests
PostgreSQL	System of record incl. chunk text (so Qdrant can be rebuilt)	Vector search
Qdrant	Dense+sparse vectors, payload filters for tenant/ACL/searchability	Being the source of truth for permissions
Redis	arq queue, rate-limit counters, stream-cancel flags, short-lived caches	Durable data
LLM API (Anthropic / OpenAI)	Answer generation, message routing + query rewriting, LLM-as-judge in evals	Receiving anything beyond the minimum retrieved context

1.2 Architecture diagram


Client
Browser
Next.js app
frontend (Next.js)
App Router pages
React Query, SSE reader
rewrites: /api/* → backend
backend (FastAPI)
Middleware
request-id, logging,
rate limit, CORS
Auth & RBAC deps
JWT → Principal
Routers v1
auth · documents · chat ·
search
comparisons · evaluations
· analytics · users
Services
enqueue job
Redis
worker (arq)
Ingestion pipeline
parse → chunk → embed
→ upsert
Evaluation runner
Purge · ACL sync · reconcile
Repositories
org-scoped
RAG core
retrieval · rerank ·
generation · citations
File storage
local / S3
PostgreSQL
LLM provider
Anthropic / OpenAI
Embedding + reranker
models
local fastembed or API
Qdrant

1.3 Backend layering

```
api (routers, deps)  →  services  →  repositories  →  db (SQLAlchemy models)
                           ↓
                        rag (providers via protocols, injected from core/config)
```

Provider swapping is done with small `typing.Protocol` interfaces and a factory reading settings:

```
class EmbeddingProvider(Protocol):
    model_id: str; dim: int
    async def embed_documents(self, texts: list[str]) -> list[list[float]]: ...
    async def embed_query(self, text: str) -> list[float]: ...

class SparseEncoder(Protocol):      # BM25
    def encode(self, texts: list[str]) -> list[SparseVector]: ...

class VectorStore(Protocol):        # Qdrant implementation; others possible
    async def upsert(...); async def hybrid_query(...); async def set_payload(...); async def delete(...)

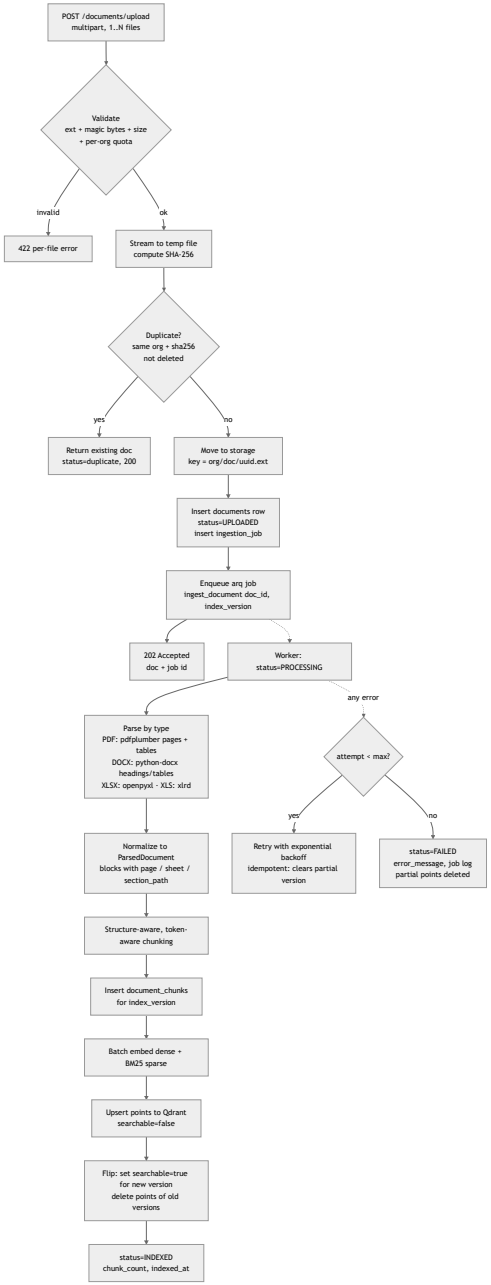
class Reranker(Protocol):
    async def rerank(self, query: str, candidates: list[Candidate], top_n: int) -> list[Candidate]: ...

class LLMProvider(Protocol):
    async def stream_grounded_answer(self, req: GroundedRequest) -> AsyncIterator[GenerationEvent]: ...
    async def complete_json(self, prompt: str, schema: type[BaseModel]) -> BaseModel: ...  # judge, routing
```

---

2. Document ingestion pipeline

2.1 Flow


POST /documents/upload
multipart, 1..N files
Validate
ext + magic bytes + size
+ per-org quota
invalid
ok
422 per-file error
Stream to temp file
compute SHA-256
Duplicate?
same org + sha256
not deleted
yes
no
Return existing doc
status=duplicate, 200
Move to storage
key = org/doc/uuid.ext
Insert documents row
status=UPLOADED
insert ingestion_job
Enqueue arq job
ingest_document doc_id,
index_version
202 Accepted
doc + job id
Worker:
status=PROCESSING
any error
Parse by type
PDF: pdfplumber pages +
tables
DOCX: python-docx
headings/tables
XLSX: openpyxl - XLS: xlrd
attempt < max?
yes
no
Retry with exponential
backoff
idempotent: clears partial
version
status=FAILED
error_message, job log
partial points deleted
Normalize to
ParsedDocument
blocks with page / sheet /
section_path
Structure-aware, token-
aware chunking
Insert document_chunks
for index_version
Batch embed dense +
BM25 sparse
Upsert points to Qdrant
searchable=false
Flip: set searchable=true
for new version
delete points of old
versions
status=INDEXED
chunk_count, indexed_at

2.2 Details

Validation. Extension allow-list (`.pdf .docx .xlsx .xls`) **and** magic-byte sniffing (`%PDF` , ZIP container with `word/` or `xl/` parts, OLE2 for `.xls`). Max size configurable (default 50 MB/file, 20 files/request). Files are streamed to disk in chunks — never fully buffered in memory. Macro-enabled formats are rejected. ZIP-bomb guard: cap total uncompressed size for OOXML.

Parsing → `ParsedDocument` **.** Each parser returns a list of `Block(text, kind=paragraph|heading|table|sheet_rows, page, sheet, section_path, bbox?)` plus document metadata (title, author, created/modified dates, page_count, sheet names).

- **PDF (pdfplumber, MIT — PyMuPDF was avoided because it is AGPL):** per-page text; `page.find_tables()` → tables rendered as Markdown and removed from the page prose so they aren't indexed twice; title/author/dates from the PDF info dictionary. Citations use page numbers (no `section_path` for PDFs; font-size heading detection is a possible later improvement). Pages with almost no text are listed in the document's `warnings` ; a PDF with no extractable text at all fails with `no_text` (OCR is out of scope for v1). Encrypted PDFs fail with `encrypted_pdf` .
- **DOCX (python-docx):** walk body in order; `Heading N` styles build `section_path` ("Security > Access Control"); tables → Markdown. No page numbers in DOCX (they're a rendering artifact) → citations use section instead.
- **XLSX/XLS:** per sheet, detect header row, emit rows as `col: value` records; computed values (not formulas) via `data_only=True` ; hidden sheets included but flagged; very large sheets capped with a warning.

Chunking (configurable per org/eval config).

1. Never cross a hard boundary: page (PDF), top-level section (DOCX), sheet (Excel).
2. Within a boundary, LangChain `RecursiveCharacterTextSplitter.from_huggingface_tokenizer` using the **embedding model's tokenizer** — sizes are true token counts for the model that embeds them. Defaults: `chunk_size=512` tokens, `chunk_overlap=64` .
3. Tables are kept whole if ≤ `chunk_size` , else split by row groups with the **header row repeated** in each chunk.
4. Excel: groups of N rows per chunk with header repeated and `sheet_name` + row range in metadata.
5. Each chunk is prefixed (for embedding only) with a short context header: `Document: <title> | Section: <path> | Sheet: <name>` — improves retrieval of out-of-context chunks. The stored `text` used for display/citation is the raw chunk.

Idempotency & searchability. Every ingestion run writes under a new `index_version` . Points are upserted with `searchable=false` ; only after *all* points are written does the worker flip them to `true` and delete older versions — a half-indexed document can never be retrieved, and reprocessing never makes a document temporarily unsearchable. Point IDs are deterministic UUIDv5(`doc_id` , `index_version` , `chunk_index`) so a retried job overwrites rather than duplicates.

Duplicates. `UNIQUE (organization_id, sha256) WHERE deleted_at IS NULL` . Re-uploading identical bytes returns the existing document (UI: "Already uploaded as …"). Same name/different bytes is a new document.

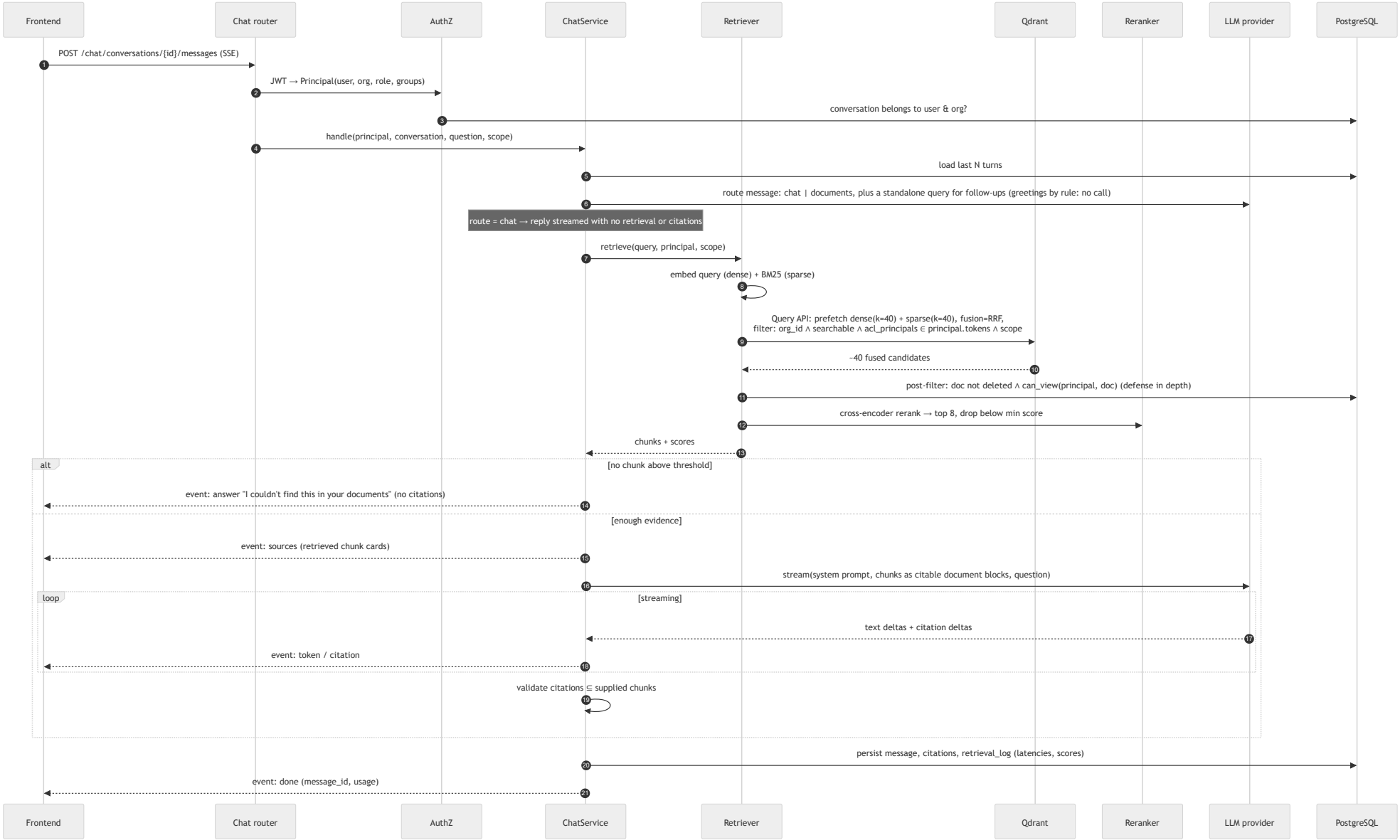
Deletion. Soft delete in PG (immediately excluded from listings and from the retrieval post-filter) → set Qdrant `searchable=false` synchronously by filter (fast) → worker purges points, chunks, and the stored file. A periodic **reconciliation job** compares PG and Qdrant counts per document and repairs drift.

Retries. arq `max_tries=3` with exponential backoff; parsing errors that are deterministic (corrupt file, encrypted PDF) fail immediately without retrying. Every attempt is recorded in `ingestion_jobs` with stage and timings.

---

3. RAG retrieval & generation pipeline

3.1 Flow


Frontend
Chat router
AuthZ
ChatService
Retriever
Qdrant
Reranker
LLM provider
PostgreSQL
POST /chat/conversations/{id}/messages (SSE)
JWT → Principal(user, org, role, groups)
conversation belongs to user & org?
handle(principal, conversation, question, scope)
load last N turns
route message: chat | documents, plus a standalone query for follow-ups (greetings by rule: no call)
route = chat → reply streamed with no retrieval or citations
retrieve(query, principal, scope)
embed query (dense) + BM25 (sparse)
Query API: prefetch dense(k=40) + sparse(k=40), fusion=RRF,
filter: org_id ∧ searchable ∧ acl_principals ∈ principal.tokens ∧ scope
~40 fused candidates
post-filter: doc not deleted ∧ can_view(principal, doc) (defense in depth)
cross-encoder rerank → top 8, drop below min score
chunks + scores
alt
[no chunk above threshold]
event: answer "I couldn't find this in your documents" (no citations)
[enough evidence]
event: sources (retrieved chunk cards)
stream(system prompt, chunks as citable document blocks, question)
loop
[streaming]
text deltas + citation deltas
event: token / citation
validate citations ⊆ supplied chunks
persist message, citations, retrieval_log (latencies, scores)
event: done (message_id, usage)

3.2 Details

> **As implemented (after Phase 7).** Routing has two routes, `chat` and `documents`, and runs before retrieval (`app/rag/routing.py`). Greetings, thanks and goodbyes are matched by rule; anything else is one structured-output call (low effort) that returns the route and, for follow-ups, the standalone query, so it replaces the separate condensation call below. A `chat` reply uses the assistant persona prompt with no document access; a failed or uncertain routing means `documents`. `summarize` and `find_documents` below remain design ideas; comparison is the Compare page.
>
> **As implemented (after Phase 7): model choice, cost, edits.** `ModelRegistry` (`app/rag/generation/registry.py`) maps a model id to a provider built on demand from the catalog `models.json`; a model is offered when its provider's key is configured, and an unknown id is rejected before anything is stored. Providers report usage in one shape (`input_tokens` is the whole prompt; cache reads/writes and reasoning tokens are separate counters; for Gemini thinking tokens are added to output), so `usage_cost()` is provider-independent; the routing call's usage is merged into the reply's total and the result is stored on the message, priced as of that day. Gemini uses Google's OpenAI-compatible endpoint through the existing `openai` SDK. Editing a user message marks it and every later message `superseded_at` (kept, hidden, excluded from context) and the replacement carries `edit_of`.

Query condensation. Follow-ups ("what about the second one?") are rewritten into a standalone query using the last few turns. The original question is still what the user sees and what goes to the final answer prompt.

Intent routing (cheap structured-output classification, with a rules fallback):

- `qa` → standard pipeline above.
- `summarize` (a specific document) → retrieval is the wrong tool for "summarize X". Use **map-reduce over all chunks of that document** (chunked summaries, then a final summary), citing the sections each summary point comes from.
- `find_documents` ("which documents mention Kubernetes?") → Qdrant `query_points_groups` grouped by `document_id`, returning a document list with best-matching snippet each.
- `compare` → see §3.4.

Hybrid retrieval. Each point has two named vectors: `dense` (cosine) and `bm25` (sparse, IDF modifier on the collection). One Query API call with two `prefetch` branches and `fusion: rrf`. The ACL/tenant filter is applied **inside each prefetch**, so unauthorized points are never candidates (not filtered after the fact).

Reranking. Cross-encoder scores (query, chunk) pairs for the ~40 fused candidates; keep top `n=8` with a minimum relevance threshold. Per-document diversity cap (max 4 chunks from one document unless the scope is a single document).

Context construction. Chunks are ordered by document then position, de-duplicated by overlap, and fitted to a token budget. Each chunk becomes one citable unit with a title like `Q3 Financial Report.pdf — p. 12 — Revenue`.

Generation (Anthropic default).

- Model `claude-opus-5-5` (configurable via `LLM_MODEL`), streaming via `client.messages.stream(...)`, `thinking: {type: "adaptive"}`, `output_config.effort` configurable (default `medium` for chat).
- Each retrieved chunk is passed as a **`document` content block with custom content and `citations: {enabled: true}`**. Claude's response then contains citation objects that reference the block index we supplied, which maps 1:1 to our `chunk_id`. A citation can only point at text we actually retrieved.

- The static system prompt is cached (`cache_control`) to cut cost/latency.
- `stop_reason == "refusal"` is handled explicitly; the server-side refusal fallback (`fallbacks: "default"`) is enabled by default and can be turned off by config.
- OpenAI adapter: same interface; chunks are numbered `[S1]…[Sn]` and the model must cite markers. Our validator maps markers back to chunks.

Citation validation (provider-independent). After generation, each citation must (a) reference a chunk we supplied in this turn and (b) its quoted text must occur in that chunk (fuzzy match ≥ 0.9). Invalid citations are dropped and logged as a quality signal. Citations are persisted with a **snippet snapshot** so history still renders even if the document is later deleted (the link then shows "document no longer available").

Facts vs. interpretation. The system prompt requires the answer to separate *document-grounded statements* (always cited) from *analysis/inference* (explicitly labelled, e.g. a final "Interpretation" section, uncited). The UI renders the two differently. If evidence is insufficient the model must say so; additionally, when retrieval returns nothing above threshold we **don't call the LLM at all**.

Prompt-injection defenses. Retrieved text is delivered as document content (data), never concatenated into the system prompt; the system prompt states that instructions inside documents must be ignored; the generation call has **no tools**, so an injected instruction cannot trigger actions; ingestion flags chunks matching injection heuristics (`ignore previous instructions` , role-play markers, hidden-text in PDFs) in payload `flags` for monitoring; output is rendered as sanitized Markdown (no raw HTML) so injected links/scripts can't execute.

3.3 Citation object (API contract)

```
{
  "index": 1,
  "chunk_id": "6f1c…",
  "document_id": "a9e2…",
  "document_title": "Security Policy v3.docx",
  "page": null,
  "page_end": null,
  "sheet": null,
  "section": "Access Control > Passwords",
  "snippet": "Passwords must be rotated every 90 days…",
  "score": 0.87,
  "url": "/documents/a9e2…?chunk=6f1c…#section"
}
```

Clicking opens the document viewer: PDFs at `#page=N` with the snippet highlighted; DOCX/Excel open the extracted-text view scrolled to the chunk (section / sheet + row range).

3.4 Document comparison

User selects 2–N documents (each must pass `can_view`). The aspects to compare come from the user question or, if generic ("compare these"), from a structure pass over each document's section titles. For each aspect, retrieval runs **per document with its own budget** (so one large document can't crowd out the others), then one generation call returns **structured output** (validated with Pydantic):

```
{ "aspects": [ { "aspect": "Data retention",
    "per_document": [ { "document_id": "…", "finding": "…", "citations": [ … ], "status":
"present|absent|unclear" } ],
    "difference_summary": "…", "conflict": true } ],
  "missing_or_inconsistent": [ … ], "overall_summary": "…" }
```

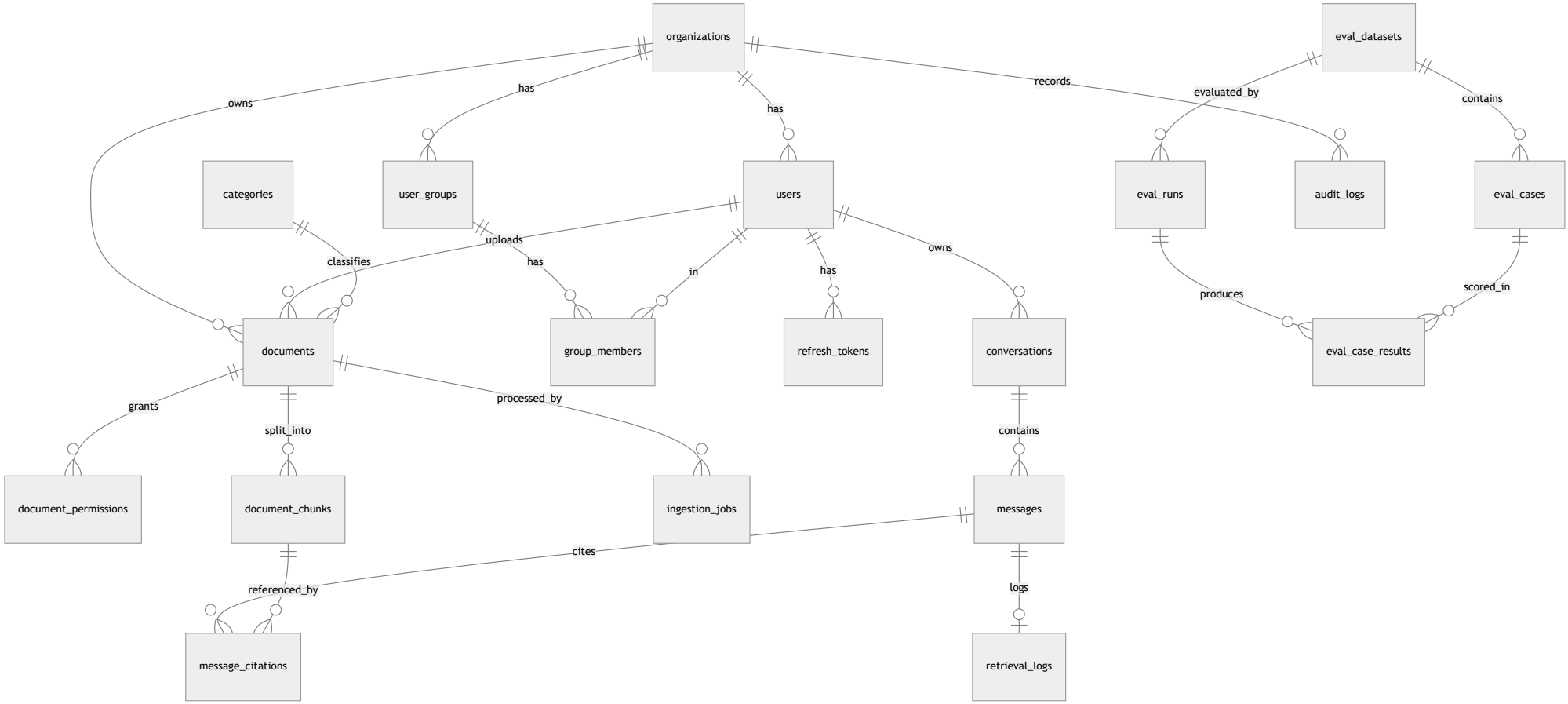
Rendered as a comparison table (aspect rows × document columns) with citations per cell.

> **As implemented (Phase 6).** Sources are numbered across documents and the model refers to documents by label (D1…); the server maps labels back to documents, rejects citations of another document's sources, downgrades unevidenced "present" findings to "unclear", fills findings the model omitted, and replaces labels with titles in user-facing text. The structured output uses the provider's strict JSON-schema mode (`complete_json`) for both Anthropic and OpenAI. Comparisons are stored (`document_comparisons`) and listed per user.

---

4. PostgreSQL schema

All PKs are UUIDv7 (time-ordered, index-friendly). All tenant tables carry `organization_id` with a composite index leading on it. Timestamps are `timestamptz`.


organizations
eval_datasets
owns
has
has
records
evaluated_by
contains
categories
user_groups
users
eval_runs
audit_logs
eval_cases
uploads
classifies
has
in
has
owns
produces
scored_in
documents
group_members
refresh_tokens
conversations
eval_case_results
grants
split_into
processed_by
contains
document_permissions
document_chunks
ingestion_jobs
messages
cites
referenced_by
logs
message_citations
retrieval_logs

4.1 Tables

Identity & tenancy

Table	Columns (key ones)	Constraints / indexes
`organizations`	id, name, slug, settings jsonb (chunking, retrieval, quotas), storage_quota_bytes, created_at	`slug` unique
`users`	id, organization_id, email (citext), password_hash, full_name, role (`admin` | `manager` | `user`), is_active, last_login_at, created_at, updated_at	`email` unique; idx (organization_id, role)
`refresh_tokens`	id, user_id, token_hash (sha256), family_id, expires_at, revoked_at, replaced_by_id, user_agent, ip, created_at	idx token_hash; reuse of a revoked token revokes the whole family
`user_groups`	id, organization_id, name, description	unique (organization_id, name)
`group_members`	group_id, user_id	PK (group_id, user_id)

Documents

Table	Columns	Constraints / indexes
`categories`	id, organization_id, name, color	unique (organization_id, name)
`documents`	id, organization_id, owner_id, category_id?, title, original_filename, mime_type, file_ext, size_bytes, sha256, storage_key, status (`uploaded` | `processing` | `indexed` | `failed`), visibility (`private` | `organization` | `restricted`), active_index_version int, chunk_count, page_count?, sheet_names text[]?, metadata jsonb (author, created, modified, warnings), embedding_config_id, error_message?, created_at, updated_at, indexed_at?, deleted_at?	**unique (organization_id, sha256) where deleted_at is null**; idx (organization_id, status), (organization_id, created_at desc), (organization_id, category_id); trigram/GIN on title for search
`document_permissions`	id, document_id, principal_type (`user` | `group`), principal_id, permission (`view` | `manage`), granted_by, created_at	unique (document_id, principal_type, principal_id)
`ingestion_jobs`	id, document_id, index_version, status (`queued` | `running` | `succeeded` | `failed`), attempt, stage (`parse` | `chunk` | `embed` | `upsert` | `finalize`), error?, metrics jsonb (timings, token counts), started_at, finished_at	idx (document_id, created_at desc)
`document_chunks`	id (= Qdrant point id), document_id, organization_id, index_version, chunk_index, text, token_count, page_start?, page_end?, sheet_name?, row_start?, row_end?, section_path?, content_hash, flags text[]	unique (document_id, index_version, chunk_index); idx (organization_id)
`embedding_configs`	id, name, embedding_provider, embedding_model, dim, sparse_model, chunk_size, chunk_overlap, qdrant_collection, is_active, created_at	one active per deployment; the basis of re-index & regression runs

Conversations

Table	Columns	Notes
`conversations`	id, organization_id, user_id, title, scope jsonb (`{document_ids?, category_ids?}`), created_at, updated_at, deleted_at?	Owner-only (admins can't read others' chats by default — privacy)
`messages`	id, conversation_id, role (`user` | `assistant`), content, status (`streaming` | `complete` | `error` | `cancelled`), intent?, model?, usage jsonb (input/output/cache tokens), latency_ms?, created_at	idx (conversation_id, created_at)
`message_citations`	id, message_id, citation_index, chunk_id? (nullable: chunk may be purged), document_id?, document_title, page?, sheet?, section?, snippet, score	Snapshot so history survives deletions
`document_comparisons`	id, organization_id, user_id, document_ids uuid[], question, result jsonb, status, created_at	

Observability & analytics

Table	Columns	Notes
`retrieval_logs`	id, organization_id, user_id, message_id?, mode (`semantic` | `hybrid`), query_hash, top_k, candidate_count, returned_chunk_ids uuid[], top_score, no_answer bool, latency jsonb (`embed, search, rerank, llm_first_token, llm_total`), embedding_config_id, created_at	Query text stored only in `messages` , never in app logs; idx (organization_id, created_at)
`audit_logs`	id, organization_id, actor_id?, action (`auth.login` , `document.delete` , `permission.grant` , `user.role_change` , …), resource_type, resource_id, ip, user_agent, details jsonb, created_at	Append-only (no UPDATE/DELETE grant for the app role)

Evaluation

Table	Columns	Notes
`eval_datasets`	id, organization_id, name, description, created_by, created_at	
`eval_cases`	id, dataset_id, question, expected_answer, relevant_document_ids uuid[], relevant_chunk_ids uuid[], evidence jsonb (`[{document_id, page?, section?, quote}]`), tags text[]	`evidence` makes ground truth survive re-chunking (see §10)
`eval_runs`	id, dataset_id, status, config_snapshot jsonb (embedding config, chunking, top_k, fusion, reranker, LLM model/effort, prompt version, git sha), baseline_run_id?, aggregate_metrics jsonb, started_at, finished_at, triggered_by	
`eval_case_results`	id, run_id, case_id, retrieved_chunk_ids uuid[], answer, citations jsonb, metrics jsonb (`p@k, r@k, mrr, ndcg, faithfulness, answer_relevance, context_relevance, citation_accuracy, hallucination`), judge_rationale jsonb, latency_ms, cost_usd	

---

5. Qdrant collection & payload design

Collection naming: `dm_<model>_<size>-<overlap>_<hash8>` (e.g. `dm_bge-base-en-v1-5_400-60_e5c51e41` ; the hash covers model, sparse model and chunking), with alias **`chunks_active`** pointing at the live one. Names are deliberately short: Qdrant nests several directory levels per collection and the Windows build is limited to 260-

character paths. Re-embedding or re-chunking builds a new collection in the background; switching is an atomic alias swap; eval runs can target any collection explicitly.

Vectors:

```
{
  "vectors":        { "dense": { "size": 768, "distance": "Cosine", "on_disk": true } },
  "sparse_vectors": { "bm25":  { "modifier": "idf" } },
  "hnsw_config":    { "payload_m": 16, "m": 0 },
  "quantization_config": { "scalar": { "type": "int8", "always_ram": true } }
}
```

`m: 0` + `payload_m: 16` builds the HNSW graph **per tenant** (multitenancy pattern), so filtered search inside one org stays fast and accurate.

Payload (per point):

```
{
  "org_id": "uuid",
  "document_id": "uuid",
  "index_version": 3,
  "chunk_id": "uuid",
  "chunk_index": 17,
  "searchable": true,
  "acl": ["admin:<org_id>", "user:<org_id>:<owner_uid>", "org:<org_id>", "group:<org_id>:<gid>"],
  "category_id": "uuid|null",
  "file_type": "pdf|docx|xlsx|xls",
  "document_title": "Q3 Financial Report.pdf",
  "page_start": 12, "page_end": 12,
  "sheet_name": null, "row_start": null, "row_end": null,
  "section_path": "Revenue > Regional",
  "text": "…raw chunk text…",
  "token_count": 412,
  "content_hash": "sha256",
  "flags": [],
  "created_at": "2026-11-06T10:00:00Z"
}
```

ACL tokens are organization-qualified, so they can never match across tenants even without the `org_id` condition (implemented in `app/services/access_policy.py`; equivalence with the Python policy is tested exhaustively).

Payload indexes: `org_id` (keyword, `is_tenant: true`), `document_id` (keyword), `acl` (keyword), `searchable` (bool), `category_id` (keyword), `file_type` (keyword), `created_at` (datetime).

Standard retrieval filter:

```
{ "must": [
    { "key": "org_id",     "match": { "value": "<org>" } },
    { "key": "searchable", "match": { "value": true } },
    { "key": "acl",        "match": { "any": ["org:<org>", "user:<org>:<uid>", "group:<org>:<g1>", "admin:
<org> (admins only)"] } }
  ],
  "must_not": [],
  "should": []
}
```

plus optional `document_id ∈ scope` / `category_id ∈ …` from the conversation scope.

---

6. RBAC & document permission strategy

6.1 Roles (org-scoped)

Capability	Admin	Manager	User
Manage users, roles, groups	✅	—	—
Org settings, embedding configs, re-index	✅	—	—
Upload documents	✅	✅	config flag (default off)
Edit/delete/reprocess/share a document	✅ any	✅ own + docs granted `manage`	docs granted `manage`
View document / retrieve its chunks	✅ all in org	per ACL	per ACL
Chat, search, compare	✅	✅	✅ (over authorized docs only)
Evaluations (run/manage)	✅	view	—
Analytics	✅ org-wide	✅ own/managed docs	own usage only
Audit log	✅	—	—

Roles map to **permissions strings** (`documents:upload` , `users:manage` , `evaluations:run` , …) checked by a FastAPI dependency: `Depends(require_permission("documents:upload"))` . Endpoints never check role names directly.

6.2 Document-level access

A principal can **view** a document iff it's in the same org **and** one of:

- principal is Admin, or the owner;
- `visibility = organization` ;
- `visibility = restricted` and a `document_permissions` row grants the user or one of the user's groups.

`private` = owner (+ admins) only. **Manage** requires owner, Admin, or an explicit `manage` grant.

6.3 Enforcement in three layers (never only the frontend)

1. **API layer:** `get_principal` dependency decodes JWT → loads user (active check) → `Principal(user_id, org_id, role, group_ids, permissions)` . Document endpoints call a single `DocumentAccessPolicy.can_view/can_manage` . Lookups of another org's IDs return **404, not 403** (no existence leak).

2. **Repository layer:** all repositories require `org_id`; there is no unscoped query method. A `visible_documents(principal)` SQL predicate is reused by listings, search, analytics.
3. **Retrieval layer:** the Qdrant filter is built **only** from the server-side `Principal` (never from request params) and applied inside the vector search. Then a **PG post-check** of the top candidates (`can_view` on their document IDs, not deleted) — a stale payload can only ever make results *narrower*, never leak.

ACL sync. On share/unshare, visibility change, group membership change or user deactivation, the service writes PG in a transaction and enqueues an `acl_sync` job that recomputes `acl` for affected documents via `set_payload` by `document_id` filter. Revocations also take effect **immediately** thanks to the PG post-check, even before the payload sync runs.

Conversations are owner-only; each new message re-evaluates permissions, so a document revoked mid-conversation stops appearing in subsequent answers. Previously stored citation snippets remain in that user's own history (they were authorized at the time); if your policy requires redaction on revocation, we can hide snippets for now-unauthorized documents at read time — see open questions.

---

7. API design (v1)

Conventions: JSON; `Authorization: Bearer <access>`; errors `{"error": {"code": "document_not_found", "message": "...", "details": {...}, "request_id": "..."}}`; list endpoints `?page=&page_size=` (max 100) returning `{items, total, page, page_size}`; timestamps ISO-8601 UTC; all IDs UUID. Rate limits per user + per IP (stricter on auth and chat). OpenAPI at `/docs`, `/openapi.json`.

Auth

Method	Path	Body / notes	Auth
POST	`/api/v1/auth/register`	`{email, password, full_name, organization_name}` → creates org + Admin. (Joining existing orgs is via admin invite.)	public, rate-limited
POST	`/api/v1/auth/login`	`{email, password}` → `{access_token, expires_in, user}`; sets `refresh_token` httpOnly/Secure/SameSite=Strict cookie	public, rate-limited, lockout on brute force
POST	`/api/v1/auth/refresh`	cookie → new access + rotated refresh	cookie
POST	`/api/v1/auth/logout`	revokes refresh family, clears cookie	any
GET	`/api/v1/auth/me`	current user, org, role, permissions	user
POST	`/api/v1/auth/change-password`	`{current_password, new_password}`	user

Users & groups (Admin)

Method	Path	Notes
GET/POST	`/api/v1/users`	list / invite (create with temp password or invite token)
GET/PATCH/DELETE	`/api/v1/users/{user_id}`	role change, deactivate (audited)

Method	Path	Notes
GET/POST	`/api/v1/groups` · PATCH/DELETE `/groups/{id}` · PUT `/groups/{id}/members`	

Documents

Method	Path	Notes
POST	`/api/v1/documents/upload`	multipart `files[]`, optional `category_id`, `visibility` → 202 `{results: [{filename, status: accepted|duplicate|rejected, document?, error?}]}`
GET	`/api/v1/documents`	filters `q, status, category_id, file_type, owner, date_from, date_to`, sort
GET	`/api/v1/documents/{id}`	metadata + latest job + permissions summary
PATCH	`/api/v1/documents/{id}`	title, category, visibility (manage)
DELETE	`/api/v1/documents/{id}`	soft delete + async purge (manage) → 202
POST	`/api/v1/documents/{id}/reprocess`	new index_version (manage) → 202
GET	`/api/v1/documents/{id}/download`	streams original, `Content-Disposition: attachment` (view)
GET	`/api/v1/documents/{id}/preview`	PDF inline stream; DOCX/Excel → extracted structured text for viewer
GET	`/api/v1/documents/{id}/chunks`	paginated chunks (citation deep-link target, eval labeling)
GET	`/api/v1/documents/{id}/jobs`	ingestion history
GET/PUT	`/api/v1/documents/{id}/permissions`	list / replace grants (manage)
GET/POST/PATCH/DELETE	`/api/v1/categories[/{id}]`	

Chat

Method	Path	Notes
POST	`/api/v1/chat/conversations`	`{title?, scope?}`
GET	`/api/v1/chat/conversations`	own conversations, newest first
GET	`/api/v1/chat/conversations/{id}`	with messages + citations
PATCH	`/api/v1/chat/conversations/{id}`	rename, change scope

Method	Path	Notes
DELETE	`/api/v1/chat/conversations/{id}`	soft delete
POST	`/api/v1/chat/conversations/{id}/messages`	`{content, scope_override?}` ; `Accept: text/event-stream` → SSE; otherwise JSON once complete
POST	`/api/v1/chat/conversations/{id}/messages/{mid}/cancel`	stop generation
GET	`/api/v1/chat/suggestions`	suggested questions derived from recent authorized documents' titles/sections

SSE events: `meta {message_id, intent}` → `sources {chunks[]}` → `token {text}`* → `citation {citation}`* → `done {usage, latency_ms}` | `error {code, message}` .

Comparisons

POST	`/api/v1/comparisons`	`{document_ids[2..5], question?}` → **SSE progress + final structured result**
GET	`/api/v1/comparisons` · `/comparisons/{id}`	history

Search

Method	Path	Body
POST	`/api/v1/search/semantic`	`{query, top_k≤50, filters{document_ids, category_ids, file_types}, rerank?: bool}` → `{results: [{chunk, document, score}]}`
POST	`/api/v1/search/hybrid`	same + `{fusion: rrf, dense_k, sparse_k}`
POST	`/api/v1/search/documents`	document-level results (grouped) for "find documents mentioning X"

Evaluations (Admin; Manager read)

Method	Path	Notes
GET/POST	`/api/v1/evaluations/datasets` · GET/PATCH/DELETE `/datasets/{id}`	
GET/POST	`/api/v1/evaluations/datasets/{id}/cases` · PATCH/DELETE `/cases/{id}` · POST `/datasets/{id}/import` (JSONL)	
POST	`/api/v1/evaluations/run`	`{dataset_id, config_overrides?, baseline_run_id?, k: 5}` → 202 run
GET	`/api/v1/evaluations/results`	runs list; `/results/{run_id}` with per-case results and diff vs baseline
GET	`/api/v1/evaluations/metrics`	time series of aggregate metrics per dataset/config

Analytics

GET	`/api/v1/analytics/overview`	**totals: documents, indexed, failed, chunks, conversations, storage bytes, queries (7d)**
GET	`/api/v1/analytics/documents`	`?from&to&interval=day` uploads/indexed/failed series, by type/category, recent activity
GET	`/api/v1/analytics/retrieval`	latency p50/p95 by stage, no-answer rate, avg top score, citation drop rate, tokens/cost

Ops

`GET /health` (liveness), `GET /ready` (PG, Qdrant, Redis reachable).

---

8. How the components communicate

From → To	Protocol	Notes
Browser → Next.js	HTTPS	Pages + static assets
Next.js → FastAPI	HTTP via `rewrites` (`/api/*`)	Same-origin for the browser; refresh cookie is first-party. Proxy buffering disabled for SSE
Frontend data layer	TanStack Query + typed fetch client generated from OpenAPI (`openapi-typescript`)	Polling `GET /documents?status=processing` every 3 s while any doc is processing (simple, reliable); SSE for chat
Streaming chat	`fetch` POST + `ReadableStream` SSE parser	`EventSource` can't POST or send `Authorization`
FastAPI → PostgreSQL	asyncpg via SQLAlchemy async, pooled	Request-scoped `AsyncSession` , unit-of-work per request
FastAPI/worker → Qdrant	`qdrant-client` AsyncQdrantClient (gRPC)	API key in prod
FastAPI → Redis	`redis.asyncio`	enqueue arq jobs, rate limits, cancel flags
Worker ↔ Redis	arq	job payload = IDs only (never document content)
Backend → LLM	`anthropic` SDK (HTTPS, streaming) / `openai` SDK	Keys only in backend env; timeouts + retries; per-org token accounting
Backend → embedding/reranker	in-process ONNX (fastembed) or provider SDK	Models cached in a volume
Backend → storage	filesystem / S3 API	Signed internal access only; downloads streamed through the API after authZ

Docker Compose services: `frontend` , `backend` , `worker` , `postgres` , `qdrant` , `redis` (+ optional `minio`). Config via environment (`pydantic-settings`), `.env.example` documents every variable; secrets never baked into images.

---

9. Security & production readiness (summary)

- Argon2id (`pwdlib`), password policy, login throttling + temporary lockout.

- JWT HS256/RS256 with `iss`, `aud`, `exp`, `jti`, `org`, `role` claims; role/active status re-checked from DB per request (cheap, cached 30 s) so deactivation is immediate.
- Refresh-token rotation with reuse detection; logout revokes the family.
- Rate limiting (Redis sliding window): auth, upload, chat, search separately.
- Upload hardening: allow-list + magic bytes + size + ZIP-bomb guard; files stored under random keys outside any web root; served only via authorized streaming with `Content-Disposition` and `X-Content-Type-Options: nosniff`.
- Structured JSON logging (structlog) with request IDs; **redaction** — no document text, prompts, tokens, or passwords in logs.
- Audit logs for: login/logout/failed login, user & role changes, uploads, deletes, reprocess, permission changes, downloads, evaluation runs, settings changes.
- Security headers (CSP, HSTS in prod), CORS locked to the frontend origin.
- Prompt-injection defenses as in §3.2.
- Tests explicitly covering cross-tenant and cross-user leakage at the API **and** retrieval layers.

> **As implemented (Phase 7).** - **Proxy chain:** client → Caddy (TLS; replaces any client `X-Forwarded-For`) → Next.js (forwards it unchanged) → FastAPI. Uvicorn trusts the header only because the backend sits on an internal network that only the frontend can reach. - **Login rate limits:** per IP and also per target account, so spoofing or rotating IPs doesn't bypass them. - **Production guards:** the API refuses to start with non-secure cookies, wildcard CORS, the example JWT secret, or no Qdrant API key. API docs are off in production. - **Content-Security-Policy:** production-only. It needs `'unsafe-inline'` for scripts, because nonces would require a Next.js proxy, which also caps upload bodies. It allows `blob:` for the PDF preview. The e2e suite fails on any CSP violation. - **Evaluations:** results and cases are redacted per viewer for documents the viewer cannot see. - **Ops details** (Docker stack, backups, upgrades, scaling) are in deployment.md.

---

10. Evaluation module design

Ground truth that survives re-chunking. Chunk IDs change whenever chunking changes, which would invalidate a chunk-ID-only dataset. Each case stores `relevant_document_ids`, optional `relevant_chunk_ids` (valid for one embedding config), and `evidence` quotes (`{document_id, page/section, quote}`). At run time, evidence quotes are resolved to the chunks *of the config under test* by text containment/overlap → per-config relevance labels. A labeling UI lets admins pick chunks from search results to create cases.

Metrics.

- Retrieval (deterministic, own code, unit-tested against hand-computed examples): Precision@K, Recall@K, MRR, NDCG@K (binary or graded relevance), plus document-level hit rate.
- Answer quality (LLM-as-judge with structured outputs and fixed, versioned rubrics; temperature-free models so we run each judge once and record rationale):
  - *Faithfulness* — claims extracted from the answer; fraction supported by the provided context.
  - *Answer relevance* — does it address the question (1–5, normalized).
  - *Context relevance* — fraction of retrieved chunks relevant to the question.
  - *Citation accuracy* — fraction of citations whose cited chunk supports the sentence it's attached to (+ fraction of cited chunks in the ground-truth set).
  - *Hallucination rate* — unsupported factual claims / total claims; plus "should have abstained" detection for unanswerable cases.
  - *Answer correctness* vs. `expected_answer` (semantic agreement).

- Ops: latency per stage, tokens, cost per case.

Runner. arq job iterates cases with bounded concurrency, uses the **same retrieval/generation code path** as chat (with a synthetic principal scoped to the dataset's documents), stores per-case results and aggregates, and compares to a baseline run. A CLI (`python -m app.rag.evaluation.run --dataset … --fail-under ndcg=0.70 --max-drop 0.03`) enables CI regression gating when embedding model, chunking, retrieval config, prompts or LLM change. The dashboard shows metric trends per config and per-case diffs (regressions highlighted).

> **As implemented (Phase 6).** Cases are evaluated sequentially as the user who started the run (so permissions apply exactly as in chat). The CLI is `python -m app.cli evaluate --dataset … [--baseline …] [--fail-under metric=value] [--max-drop 0.05]` . The baseline defaults to the dataset's previous completed run; `hallucination_rate` and latency are lower-is-better, latency is reported but never gated. Citation accuracy is measured deterministically (cited chunk ∈ relevant chunks, or relevant documents when no evidence quote is given); the judge returns faithfulness/hallucination, answer and context relevance, correctness and abstention with a stored rationale (`judge-v2` : same output schema as v1; the rubric now ignores friendly wording and defines the 1-5 scales). The UI covers datasets, cases (form or JSON import), runs with per-case answers/citations/rationale, baseline deltas and a metric trend.

---

11. Technical challenges & recommended solutions

#	Challenge	Recommended solution
1	**Leakage across tenants/users**	Filter inside vector search from server-side principal + PG post-check; org-scoped repositories; 404 for foreign IDs; dedicated leakage test suite
2	ACL changes vs. denormalized Qdrant payload	PG is truth; post-check makes revocations immediate; async `set_payload` sync; nightly reconciliation
3	Partially indexed documents becoming searchable	`searchable` flag flipped only after full upsert; versioned point IDs; idempotent retries
4	PG ↔ Qdrant drift (crash between writes)	Ordered writes (PG chunks → Qdrant → flip → PG status), deterministic IDs, reconciliation job, Qdrant rebuildable from `document_chunks`
5	Scanned / image-only PDFs	Detect low-text pages, warn in UI; OCR (Tesseract/ `ocrmypdf`) as an optional worker stage later
6	Tables in PDFs & huge spreadsheets	Table extraction to Markdown; header-repeating row-group chunks; caps with warnings; "extract figures" queries favor table chunks via payload `kind`
7	"Summarize this document" / "which documents mention X" don't fit top-K RAG	Intent routing: map-reduce summarization over the whole doc; grouped document search
8	Comparisons dominated by one large document	Per-document retrieval budgets; structured output per aspect
9	Follow-up questions	Query condensation from recent turns; scope carried in conversation
10	Fabricated or wrong citations	Native citation blocks (Anthropic) / marker mapping (OpenAI) + server-side quote validation; drop invalid; track drop rate as a metric
11	Answering without evidence	Rerank-score threshold → abstain without calling the LLM; prompt requires explicit "not found"; eval includes unanswerable cases

#	Challenge	Recommended solution
12	Prompt injection in documents	Documents as data blocks, no tools on generation, heuristic flags, sanitized rendering
13	Eval ground truth invalidated by re-chunking	Evidence-quote–based labels resolved per config
14	Changing embedding model / chunking	`embedding_configs` + collection-per-config + alias swap; background re-index; eval before switching
15	Latency (embed + search + rerank + first token)	Local ONNX models kept warm, gRPC Qdrant, int8 quantization, prompt caching, stream sources before tokens, per-stage latency logged
16	Cost control	Per-org token accounting in `messages.usage`, configurable model/effort, skip LLM on no-evidence, cache system prompt
17	SSE through proxies	`X-Accel-Buffering: no`, heartbeats every 15 s, cancel endpoint + client abort handling
18	Large uploads / memory	Streamed uploads to disk, size caps, parsing in worker only

12. Proposed repository layout

```
backend/
  app/
    api/v1/{auth,users,groups,documents,categories,chat,comparisons,search,evaluations,analytics}.py
    api/deps.py               # get_db, get_principal, require_permission
    core/{config,security,logging,rate_limit,errors}.py
    db/{base,session}.py
    models/                   # SQLAlchemy models
    schemas/                  # Pydantic request/response
    repositories/
    services/
    rag/
      ingestion/{validators,parsers/{pdf,docx,excel},pipeline}.py
      chunking/
      embeddings/{base,fastembed,openai}.py
      retrieval/{qdrant_store,hybrid,filters}.py
      reranking/{base,cross_encoder,cohere}.py
      generation/{base,anthropic,openai,prompts,citations}.py
      evaluation/{metrics,judges,runner}.py
    workers/{settings,tasks}.py
    utils/
    main.py
  tests/{unit,integration,security,rag}/
  alembic/
  pyproject.toml / requirements.txt
  Dockerfile
frontend/
  app/(auth)/login, register ·
app/(dashboard)/{page,documents,embeddings,chat,evaluations,analytics,users,settings}
  components/ui (shadcn) · components/layout · features/{documents,chat,analytics,evaluations,users}
  hooks/ · lib/{api-client,auth,sse}.ts · services/ · types/ (generated from OpenAPI) · public/
infrastructure/{docker-compose.yml, qdrant/, postgres/}
docs/architecture.md
.env.example
README.md
```

13. Open questions for approval

1. **LLM provider/model:** Anthropic `claude-opus-5-5` as default (OpenAI adapter available)? Or do you want a different default model for cost reasons?
2. **Embeddings:** local `bge-base-en-v1.5` (no extra key, English-focused) vs. an API model (OpenAI / Voyage) or multilingual `bge-m3`? Are documents multilingual?
3. **Tenancy:** one organization per user (proposed) vs. users belonging to multiple workspaces?
4. **Can the `User` role upload?** Proposed: off by default, org setting.

5. **Revocation & history:** keep previously-shown citation snippets in a user's chat history after access is revoked, or redact on read?
6. **OCR for scanned PDFs:** in scope for v1 or later?
7. **Storage:** local volume for dev + S3-compatible for prod (proposed) — do you want MinIO in docker-compose from day one?
8. **Reference UI image:** it wasn't included with the prompt — please attach it before Phase 5.

Deploying DocuMind

This guide covers running DocuMind with Docker Compose on a single Linux host. That setup suits small and mid-sized teams; the scaling section describes what to change beyond it. Commands run from the repository root. To put a demo online with the website on Vercel and the backend on AWS Lightsail (including the scripts, costs and safeguards for a public demo), follow deploy-vercel-lightsail.md.

Topology

```
browser ──HTTPS──► caddy ──► frontend (Next.js) ──/api/*──► backend (FastAPI) ──► postgres
                   :80/:443      edge + internal networks        internal only        redis
                                                                  worker (arq) ──────► qdrant
```

- **Caddy** terminates TLS and is the only service with published ports. It replaces any client-sent `X-Forwarded-For` with the real peer address.
- The **frontend** serves the UI and proxies `/api/*` to the backend, so cookies stay first-party and the browser needs no CORS.
- The **backend** and **worker** run the same image. The backend trusts `X-Forwarded-For` (`FORWARDED_ALLOW_IPS=*`) because the network layout lets only the frontend reach it. That trust is what gives correct per-IP rate limits and audit IPs. Do not publish the backend's port.
- **migrate** runs `alembic upgrade head` once, before the backend and worker start.
- **Volumes:**
  - `postgres-data` (users, documents, chunks, audit log, evaluations)
  - `file-storage` (uploaded originals)
  - `qdrant-data` (vectors)
  - `redis-data`
  - `model-cache` (downloaded embedding and reranker models)
  - `caddy-data` (certificates)

1. Prerequisites

- Docker Engine 24+ with the Compose plugin.
- At least 4 GB of RAM: the embedding and reranker models run on the CPU, and the worker holds them in memory.
- About 3 GB of disk for images and models, plus your documents.
- For a public deployment: a DNS name pointing at the host, with ports 80 and 443 open (Let's Encrypt needs both).

2. Configure

```
cp .env.example .env
python3 -c "import secrets; print(secrets.token_hex(24))"   # run once per secret below
```

Set these in `.env`:

Variable	Value
`POSTGRES_PASSWORD`	Random **hex** string. Compose builds the database URL from it, so it must be URL-safe.
`JWT_SECRET_KEY`	Random, at least 32 characters. The app refuses to start with the example placeholder.
`QDRANT_API_KEY`	Random. Required in production because Qdrant stores document text.
`SITE_ADDRESS`	Your domain (e.g. `docs.example.com`), or `localhost` for a trial run.
`LLM_PROVIDER`, `LLM_MODEL`, `OPENAI_API_KEY` / `ANTHROPIC_API_KEY` / `GOOGLE_API_KEY`	The default chat model and the provider keys. Every provider with a key adds its models to the chat model picker. Search and ingestion work without any. To change the offered models or prices, mount your own `models.json` into the backend container and set `LLM_CATALOG_FILE`.
`HSTS_MAX_AGE`	`63072000` once the site runs on its real domain with a valid certificate.

Compose forces the production settings: `ENVIRONMENT=production`, secure cookies, JSON logs, CORS limited to `https://$SITE_ADDRESS`, and API docs off. In production the app also refuses to start when the secrets are insecure.

3. Start

```
docker compose -f infrastructure/docker-compose.yml --env-file .env up -d --build
docker compose -f infrastructure/docker-compose.yml --env-file .env ps     # all "healthy"
```

Open `https://$SITE_ADDRESS`. The first visitor registers a new organization and becomes its admin.

On the first document upload, the worker downloads its models into `model-cache`. That takes a few minutes; later uploads and restarts reuse the cache.

Verify with the smoke test. It registers a throwaway account, uploads and indexes a document, runs a search, and checks the security headers:

```
pip install python-docx
python3 infrastructure/smoke_test.py https://$SITE_ADDRESS --production   # add --insecure for localhost
```

With `SITE_ADDRESS=localhost`, Caddy uses its own internal CA. Browsers will warn unless you trust that CA, so use `--insecure` with the smoke test.

4. Upgrade

```
git pull
docker compose -f infrastructure/docker-compose.yml --env-file .env up -d --build
```

- `migrate` applies new database migrations before the new backend and worker start.
- Back up first (see below). Migrations are forward-only in production.
- If a release changes the embedding model or the chunking settings, it creates a new index. Search keeps using each document's current index until it is re-indexed. Re-index from the Embeddings page, or run: `docker compose ... exec backend python -m app.cli reindex --stale`.

5. Back up and restore

PostgreSQL and the uploaded files are the **source of truth**. Qdrant can be rebuilt from them with `python -m app.cli reindex --all` (this re-embeds every document). Snapshot Qdrant anyway if you want restores to be quick.

```
C="docker compose -f infrastructure/docker-compose.yml --env-file .env"
# Database (consistent online dump)
$C exec -T postgres sh -c 'pg_dump -U "$POSTGRES_USER" -Fc "$POSTGRES_DB"' > documind-$(date +%F).dump
# Uploaded files
docker run --rm -v documind_file-storage:/data:ro -v "$PWD":/backup alpine \
  tar czf /backup/documind-files-$(date +%F).tgz -C /data .
```

Restore into a fresh stack:

```
$C up -d postgres
$C exec -T postgres sh -c 'pg_restore -U "$POSTGRES_USER" -d "$POSTGRES_DB" --clean --if-exists' <
documind-YYYY-MM-DD.dump
docker run --rm -v documind_file-storage:/data -v "$PWD":/backup alpine \
  tar xzf /backup/documind-files-YYYY-MM-DD.tgz -C /data
$C up -d
$C exec backend python -m app.cli reindex --all     # rebuild vectors if qdrant-data was not restored
```

Store backups off the host and encrypt them: they contain every document.

6. Operate

- **Health:** inside the network, the backend serves `/health` (liveness) and `/ready` (PostgreSQL, Redis and Qdrant reachable); the worker answers `arq --check` (heartbeat). Compose uses all of these for its health checks.
- **Logs:** `$C logs -f backend worker` prints one JSON line per event. Each request is logged with its `request_id`, which is also returned in the `X-Request-ID` header and in every error body. Logs never contain document text, prompts, tokens or passwords.
- **Audit:** the `audit_logs` table records logins (including failed ones), user and role changes, uploads, downloads, deletions, sharing changes, settings changes and evaluation runs.
- **Failed ingestion:** see the document's *Details* tab, or retry everything with `$C exec backend python -m app.cli reindex --failed`. A sweep job also recovers lost or stuck jobs every 5 minutes.
- **Quality regressions:** run an evaluation dataset after changing models, chunking or prompts: `$C exec backend python -m app.cli evaluate --dataset <id> --max-drop 0.05`. It exits with 1 on a regression.

Scaling

- **More ingestion throughput:** `$C up -d --scale worker=3`. Jobs are distributed through Redis.
- **API replicas:** the backend is stateless; sessions live in PostgreSQL and rate limits in Redis. Run more replicas behind the frontend, or move to an orchestrator.
- **Managed datastores:** point `DATABASE_URL`, `REDIS_URL` and `QDRANT_URL` at managed services and remove those services from the compose file.

- **Another proxy or load balancer in front of Caddy:** add its addresses to `trusted_proxies` in `infrastructure/caddy/Caddyfile`. Otherwise every request will appear to come from the load balancer.

Security checklist

- [ ] Secrets are random and unique, and `.env` is readable only by the deploying user (`chmod 600 .env`).
- [ ] Only ports 80 and 443 are reachable from outside; the datastores have no published ports.
- [ ] `HSTS_MAX_AGE` is set once HTTPS works on the real domain.
- [ ] Backups run regularly, are encrypted, are stored off the host, and a restore has been tested.
- [ ] The host OS and Docker are kept updated, and images are rebuilt regularly for base-image fixes.
- [ ] LLM provider data handling is acceptable for your documents. With `LLM_PROVIDER=openai`, requests are sent with `store=false`.

Deploying DocuMind: website on Vercel, backend on AWS Lightsail

This is a step-by-step guide for putting a working copy of DocuMind online for a client or a portfolio:

```
visitor ──HTTPS──▶ Vercel (the Next.js website)
                     │  /api/*  is proxied to your server
                     ▼
                AWS Lightsail server (Docker Compose, HTTPS by Caddy)
                caddy ─▶ frontend ─▶ backend ─▶ postgres · redis · qdrant
                                     worker (reads documents, builds the search index)
```

Why two places? Vercel runs websites, not long-running servers. DocuMind's backend needs a worker, three datastores and local AI models (about 4 GB of RAM), so it runs on a small server. The Lightsail server also serves the complete app on its own address, which is a handy fallback.

Everything here was prepared and tested piece by piece on a Windows PC (scripts, the Vercel build, the account and model safeguards). It has **not** yet been run on a real Lightsail server, and the Docker setup has not run in CI yet, so expect to fix small things on the first deploy. If something fails, the logs (section 9) tell you what.

1. What it costs and what you need

Lightsail server	**$24 per month** for the 4 GB / 2 vCPU / 80 GB plan (pricing). **The small plans do not work**: 512 MB ($5), 1 GB ($7) and 2 GB ($12) cannot hold the AI models, and even building the Docker images runs out of memory. AWS bills by the hour up to the monthly price, so a 3-day demo costs a few dollars.
Static IP	Included while attached to the instance. **Release it when you delete the instance**: an unattached one is billed.
Vercel	Free Hobby plan. It is limited to non-commercial, personal use (terms): fine for a portfolio, borderline for paid client work.
Host name	A free `duckdns.org` name, or any domain you own.
AI usage	Your OpenAI / Google keys. Costs depend on use; set a monthly limit at each provider (section 8).
On your PC	PowerShell and `scp` / `tar` (built into Windows 10/11), Node.js for the Vercel CLI.

2. Create the Lightsail server

1. Lightsail console → **Create instance**. Pick a region near you or your client.
2. Platform **Linux/Unix**, blueprint **OS Only** → **Ubuntu 24.04 LTS**.
3. Plan: the **$24** one (4 GB). Use the normal dual-stack plan, not "IPv6 only".
4. Name it `documind` and create it.
5. **Networking** → **Create static IP** and attach it to the instance. Note the address (called `SERVER-IP` below).
6. On the instance's **Networking** tab, under **IPv4 Firewall**, add **HTTP (80)** and **HTTPS (443)**. Port 80 is needed to get the HTTPS certificate. Keep SSH (22), ideally limited to your own IP.
7. **Account** → **SSH keys**: download the default key for your region (a `.pem` file).

3. Give the server a name

Caddy gets a free HTTPS certificate automatically, but only for a real host name that points at the server.

1. Sign in at duckdns.org and create a subdomain such as `documind-api`.
2. Set its **current ip** to `SERVER-IP`.
3. Check from your PC: `nslookup documind-api.duckdns.org` should print `SERVER-IP`.

The host name is then `documind-api.duckdns.org` (`HOST` below). Use your own domain instead if you have one: create an `A` record pointing at `SERVER-IP`.

4. Put the code on the server

On your PC, in the project folder:

```
powershell -ExecutionPolicy Bypass -File infrastructure\lightsail\make-bundle.ps1
```

This creates `documind-bundle.tgz` (about 0.5 MB). It contains your working files, including uncommitted changes, but no `.env`, no secrets and no `node_modules`.

Windows refuses keys that other users can read. Fix the key's permissions once:

```
$key = "$HOME\Downloads\LightsailDefaultKey-ap-south-1.pem"     # your key file
icacls $key /inheritance:r /grant:r "${env:USERNAME}:R"
scp -i $key documind-bundle.tgz ubuntu@SERVER-IP:~/
```

5. Install and start everything

Connect (Lightsail's orange **Connect using SSH** button works, or `ssh -i $key ubuntu@SERVER-IP`), then:

```
mkdir -p documind && tar -xzf documind-bundle.tgz -C documind && cd documind
sudo bash infrastructure/lightsail/bootstrap-host.sh      # one time: Docker, 4 GB swap, log limits
exit                                                      # log in again so the docker group applies
```

Reconnect and deploy:

```
cd documind
bash infrastructure/lightsail/deploy.sh HOST --smoke
```

The script:

1. Creates `.env` with new random secrets and **demo-safe defaults**: sign-up closed (`REGISTRATION_ENABLED=false`) and only cheap models selectable (`CHAT_MODEL_IDS=gpt-5-nano,gpt-5-mini,gemini-3.5-flash-lite`).
2. Asks for your OpenAI and/or Google key (typed input is hidden) and stores them in `.env` (readable only by you).
3. Builds and starts the stack. The first build takes **5 to 10 minutes**.
4. Waits until `https://HOST/login` answers, which also means the certificate was issued.

5. Offers to **create your admin account** (the only way in, as sign-up is closed).
6. With `--smoke`, uploads a test document and checks search. The first run downloads the AI models (about 1.5 GB), so allow several more minutes.

Then open `https://HOST`, sign in and try it: upload a document, ask a question. **This address is already a complete, working DocuMind.**

6. Put the Vercel website in front

You only need this if you want the Vercel address. The fastest way needs no GitHub:

```
cd frontend
npx vercel login
npx vercel link                            # create a project, e.g. "documind"
npx vercel env add API_URL production      # value: https://HOST   (no trailing slash)
npx vercel --prod
```

Or, from the Vercel dashboard: **Add New** → **Project**, import the GitHub repository, set **Root Directory** to `frontend`, and add the environment variable `API_URL = https://HOST`.

`API_URL` is read at build time. If you forget it, the build stops with a message telling you to set it. After changing it, redeploy.

Vercel prints the address (`https://documind-xxxx.vercel.app`). Open it, sign in, chat, upload.

Things to know about this setup:

- **Visitors share one IP address** as far as the server can tell: it sees Vercel's servers, not the visitor. The per-IP login limit (10 a minute) is therefore shared by everyone. The per-account limit still works. For a demo this is fine; if people get "too many requests", raise `RATE_LIMIT_AUTH_PER_MINUTE` in `.env`.
- **Large uploads**: Vercel documents a 4.5 MB request limit for its functions but does not say whether it applies to proxied `/api` calls. If uploading a bigger file through the Vercel address fails, upload through `https://HOST` instead, or use small files for the demo.
- If Vercel shows a login wall, switch off **Deployment Protection** for production in the project settings.

7. Updating later

```
powershell -ExecutionPolicy Bypass -File infrastructure\lightsail\make-bundle.ps1
scp -i $key documind-bundle.tgz ubuntu@SERVER-IP:~/
```

```
tar -xzf documind-bundle.tgz -C documind && cd documind && bash infrastructure/lightsail/deploy.sh HOST
```

Your `.env`, data and uploaded files are kept. For website changes also run `npx vercel --prod` in `frontend`.

Add more people from **User Management** in the app, or create another workspace on the server:

```
docker compose -f infrastructure/docker-compose.yml --env-file .env exec backend \
  python -m app.cli create-admin --email someone@example.com --name "Some One" --organization "Their Org"
```

8. Keep it safe and cheap

- **Sign-up stays closed.** Anyone who could register could use your API keys. Share logins only with people who should have them, and change the demo password afterwards (Settings → Password).
- **Set a monthly spending limit with each AI provider** (OpenAI: Settings → Limits; Google AI Studio / Cloud: budget alerts). Better: create separate keys just for the demo.
- **Cheap models only.** `CHAT_MODEL_IDS` limits what users can pick. Change it in `.env`, then run `deploy.sh`.
- **Restrict SSH** (port 22) to your own IP in the Lightsail firewall.
- **To stop paying**: take a snapshot (about $0.05 per GB-month, roughly $4 a month for the whole disk), delete the instance, and **release the static IP**. Restore later from the snapshot. Or just delete everything and redeploy (about 15 minutes), as the demo data is easy to recreate.

9. When something goes wrong

Run these on the server, in the `documind` folder:

```
docker compose -f infrastructure/docker-compose.yml --env-file .env ps                # who is healthy
docker compose -f infrastructure/docker-compose.yml --env-file .env logs --tail 80 backend worker caddy
free -h                                                                                # memory and swap
```

Symptom	Likely cause and fix
`deploy.sh` waits and then says the site did not come up	The host name does not point at the server yet (`nslookup HOST`), or ports 80/443 are closed in the Lightsail firewall. Caddy's log (`logs caddy`) names the certificate error.
`permission denied` talking to Docker	Log out and back in after `bootstrap-host.sh`, or prefix the command with `sudo`.
A container restarts, `Killed` in the logs, `free -h` shows no memory	Out of memory. Check the swap file exists (`swapon --show`), or move to the 8 GB plan ($44).
Chat says "Answer generation is not configured"	No API key in `.env`. Add `OPENAI_API_KEY=` or `GOOGLE_API_KEY=`, then run `deploy.sh HOST` again.
A model is greyed out in the picker	That provider's quota or spending limit is used up; the message says so.
Vercel build fails with "API_URL is not set"	Add the variable in the Vercel project settings (Environment Variables) and redeploy.
The Vercel site loads but sign-in or chat fails with 502/504	Vercel cannot reach `https://HOST`: check the certificate is valid (open it in a browser) and the stack is up.
"Too many requests" at sign-in	Everyone shares Vercel's IP for the per-IP limit (section 6). Wait a minute or raise `RATE_LIMIT_AUTH_PER_MINUTE`.
First upload is slow	The AI models download on first use (about 1.5 GB). Later uploads reuse them.

If you cannot solve it, copy the output of the three commands above; that is what is needed to diagnose it.